

Software Design Document

2IPE0 SOFTWARE ENGINEERING PROJECT

PROJECT TEAM

S.A.M. Baggen	1295632
A. Bilbas	1246763
S.W. van den Broek	1324268
Y. Cui	1294504
M. van Gool	1333062
W. Haneveer	1300334
P.J. Heijstek	1316249
S.O.A. Jameel	1339567
P.P.H. van Leeuwen	1303422
M. van de Looij	1316842
J.R. Oerlemans	1338943

PROJECT MANAGERS

Y. Georgiev
S. Zarkova

PROJECT SUPERVISOR

H.T. de Beer

CLIENT

M. Campobasso



Stealthy Adaptable Machine

ABSTRACT

This document is the Software Design Document (SDD) for SAM, developed by Crawling Dummies. SAM is a robust and stealthy web crawler that tries to generate web traffic similar to human-generated web traffic. SAM is developed as part of the Software Engineering Project at the Eindhoven University of Technology. The design decisions presented in this document comply with the requirements as stated in the User Requirements Document (URD) [1]. This document complies with the European Space Agency (ESA) standards [2].

Contents

1	Introduction	7
1.1	Purpose	7
1.2	Scope	7
1.3	List of definitions	8
1.3.1	Definitions	8
1.3.2	Abbreviations and Acronyms	10
1.4	List of references	10
1.5	Overview	11
2	System Overview	12
2.1	Relation to current projects	12
2.2	Relation to predecessor and successor projects	12
2.3	Function and purpose	12
2.4	Environment	13
2.4.1	Operating System	13
2.4.2	MongoDB	13
2.4.3	Docker	14
2.4.4	Tor	14
2.4.5	The dark web	14
2.4.6	User	14
2.5	Relation to other systems	14
3	System Architecture	15
3.1	Architectural design	15
3.1.1	Front-end	16
3.1.1.1	GUI module	16
3.1.1.2	CLI module	16
3.1.2	Back-end	18
3.1.2.1	Server side	18
3.1.2.1.1	Crawler module	18
3.1.2.1.2	Interpreter module	18
3.1.2.1.3	Trainer module	20
3.1.2.2	Utility side	20
3.1.2.2.1	Enumeration	20
3.1.2.2.2	Configuration	20
3.1.3	Database module	20
3.2	Logical model description	21
3.2.1	Crawler module	22
3.2.2	Interpreter module	22
3.2.3	Trainer module	24
3.2.3.1	Training methods	25
3.2.4	Configuration module	26
3.2.5	GUI module	26
3.2.6	CLI module	27
3.2.7	Database module	28
3.2.7.1	The connection to the database	28
3.2.7.2	The custom API	29
3.2.7.3	The Query Builder	29
3.3	State dynamics	30
3.3.1	Inserting keywords	30
3.3.2	SAM starts and is trained on the first web page	31
3.3.3	Pausing SAM	32
3.3.4	Terminating SAM	33
3.3.5	Generating a schedule	34
3.3.6	Crawler log in	35
3.3.7	Crawler storing a relevant web page	36
3.3.8	Crawler asks user to solve CAPTCHA	37
3.4	Data model	38

3.5	External interface definitions	39
3.5.1	Tor browser	39
3.5.2	MongoDB database	39
3.6	Design rationale	40
3.6.1	tor-browser-selenium	40
3.6.2	Back-end language	40
3.6.3	Front-end language	40
3.6.4	Operating System	40
3.6.5	Database	41
3.6.6	Separation and modularity	41
3.6.7	Continuous integration and deployment	41
3.6.8	Packages in Python	42
4	Feasibility and Resource Estimates	44
4.1	Resource requirements	44
4.1.1	Development machine requirements	44
4.1.2	Client machine requirements	44
4.1.3	Database machine requirements	44
4.2	Performance	45
4.2.1	Performance metrics	45
4.2.2	Test setup	45
4.2.3	Results	46

DOCUMENT STATUS SHEET

GENERAL

Document title: Software Design Document

Document identifier: SDD/1.0.0

Version: 1.0.0

Authors: S.A.M. Baggen
A. Bilbas
S.W. van den Broek
Y. Cui
M. van Gool
W. Haneveer
P.J. Heijstek
S.O.A. Jameel
P.P.H. van Leeuwen
M. van de Looij
J.R. Oerlemans

Document Status: Complete

DOCUMENT HISTORY

Version	Date	Authors	Reason
0.1.0	18-05-2021	W. Haneveer M. van de Looij	Initial version.
0.2.0	31-05-2021	A. Bilbas S.W. van den Broek Y. Cui M. van Gool W. Haneveer S.O.A. Jameel M. van de Looij J.R. Oerlemans	Implementation of the sections.
0.3.0	11-06-2021	S.A.M. Baggen Y. Cui S.O.A. Jameel	Further implementation of sections and implementation of feedback.
0.4.0	28-06-2021	S.A.M. Baggen S.W. van den Broek Y. Cui P.J. Heijstek S.O.A. Jameel J.R. Oerlemans	Overall improvement of the document.
1.0.0	01-07-2021	S.W. van den Broek Y. Cui S.O.A. Jameel M. van de Looij	Final changes.

DOCUMENT CHANGE RECORDS

Version	Date	Section	Reason
0.1.0	18-05-2021	Section 1 Section 2	Initial version with layout.
0.2.0	31-05-2021	All	Implementation of the sections.
0.3.0	11-06-2021	All	Further implementation of sections and implementation of feedback.
0.4.0	28-06-2021	All	Overall improvement of the document.
1.0.0	01-07-2021	All	Final changes.

1 INTRODUCTION

1.1 PURPOSE

The Software Design Document (SDD) contains an overview of the design of Stealthy Adaptable Machine (SAM). SAM is a robust and stealthy web crawler that collects data from online underground markets and tries to be undetectable by mimicking human behaviour. The SDD aims to provide an overview of SAM's function and relation to other systems and projects. Furthermore, the SDD describes the architecture of SAM and the rationale behind this architecture. Finally, the SDD gives an estimation of the resources needed to run SAM. This document is primarily intended for professionals in the software development sector.

1.2 SCOPE

SAM is a web crawler developed by Crawling Dummies for the Software Engineering Project (SEP) of the bachelor programme Computer Science and Engineering from the Eindhoven University of Technology. Crawling Dummies is a group of bachelor students from the Eindhoven University of Technology. The aim of this project is to create a web crawler that is robust, stealthy and, if feasible, scalable. The robustness and stealthiness of the web crawler have priority over its scalability.

A robust web crawler crawls on most forums without crashing and thus quitting due to an error. All errors are communicated to the user via a Command Line Interface (CLI). When there are errors that require user interaction, they may be solved via the CLI or via a dedicated Graphical User Interface (GUI).

A stealthy web crawler crawls web pages while remaining undetected by server bot detection scripts. To remain undetected, the web crawler should mimic human behaviour while crawling platforms.

A scalable web crawler may have multiple instances of itself running simultaneously on different machines; these instances should not overwrite each other's data unintentionally.

To save the data the web crawler gathers, SAM should understand how components on a page are structured. A platform structure can be viewed as the structure of a web page belonging to a platform, which is a collection of web pages that can be accessed via the same domain name and share one structure. During the training phase, the user is able to identify the parts of the web page that are of interest, such as the title of a thread. The user can also identify the parts of the web page that are needed to navigate the forum, such as the button leading to the next page. If the user has already specified the platform structure, the user needs to confirm that the structure is correct. Afterwards, SAM autonomously crawls the platform without a lot of required user interaction, though some user interaction is still possible. For example, user interaction is necessary when the web crawler identifies a CAPTCHA it cannot solve. The user is then asked to solve these CAPCHAs.

The client will use this web crawler to gather data from underground markets in order to research these markets with SAM's undetectable behaviour. SAM aims to provide the possibility to gain valuable insights into cybercriminals that trade illicit goods in underground markets. These insights are valuable to researchers who want to analyse the behaviour of cybercriminals after having the web crawler collect the data. Undetectable in this context refers to mimicking human behaviour such that the servers hosting the crawled web pages and their administrators are not aware that SAM is a web crawler.

1.3 LIST OF DEFINITIONS

1.3.1 DEFINITIONS

Term	Description
Blacklisted keyword	A keyword indicating that the web crawler should not crawl or should abort crawling a thread containing the keyword.
Break	A scheduled pause for the web crawler to imitate, e.g., a lunch break.
CAPTCHA	A CAPTCHA, or Completely Automated Public Turing test to tell Computers and Humans Apart, is a security feature against spam, bots and DDoS attacks, intended to be solvable by humans and unsolvable by machines. There are 11 common types of CAPTCHA, namely the Simple Math Problem, Word Problem, Social Media sign-in, Time-Based, Honeypot, Picture identification, No CAPTCHA reCAPTCHA, Invisible ReCAPTCHA, Confident ReCAPTCHA, Sweet CAPTCHA and Biometrics [3].
Crawl	‘Crawl’ or ‘crawling’ is a term used to denote a bot surfing the Internet and downloading all data relevant to the bot’s purpose. For example, Google uses web crawlers to index web pages, such that their search engine is able to display them when they might be relevant [4].
Crawling session	A continuous time range in which the web crawler is planned to crawl. For example, a crawling session is the time between the start of a workday and the start of the first break (if there is one). Similarly, a crawling session is the time between the end of the last break and the end of the workday.
Cybercriminal	Cybercriminals are individuals or teams of people who use technology to commit malicious activities on digital systems or networks with the intention of stealing sensitive company information or personal data and generating profit [5].
Fetch delay	The amount of time the web crawler waits before fetching a new web page.
Illicit content	Any content of the web page that is prohibited by law. This can be various things (like credit card fraud or child pornography) in various formats (e.g. images, videos).
Interruption	A short pause that imitates, for example, a coffee break or a conversation with co-workers. When not configured by the user, an interruption is by default between 5 and 15 minutes and the exact length is randomly chosen but must be in the specified interval. Interruptions are not scheduled by the user and can happen randomly throughout sessions. The exact number of interruptions is randomly determined by the average time between interruptions, but the average time between interruptions can be configured.
Maximum thread age	The maximum age that a thread may be in order to be crawled.
Maximum thread length	The maximum number of posts to be crawled in a thread.
Navigational elements	Elements of the web page that are used to navigate the platform. These include pagination buttons, buttons to go back to the thread/section selection page and buttons that take you back to the front page.
Onion routing	Special type of routing which uses encryption to layer encryption on top of the packet like the layers of an onion. A more in-depth explanation is in Section 2.5 of the User Requirements Document (URD) [1].
Onion service	Websites hosted under *.onion top-level domain names, which are only accessible through the Tor network.

Term	Description
Platform	A collection of web pages that can be accessed via the same domain name (e.g. canvas.tue.nl).
Platform structure	The structure of a web page belonging to a platform. There may be different types of web pages, like a web page displaying the body of a thread or a web page displaying the thread selector. The locations of certain properties in the HTML file, such as thread name, number of posts, author are part of the structure. This structure is saved so that the data can easily be retrieved.
Post	The contents of a single message posted in a thread on a forum. A post is written by a single author, on a single day and may be removed or edited by the original author or forum moderators. Posts only exist within threads and cannot exist outside of threads.
Relevant keyword	A keyword that indicates that the user is interested in threads containing said keyword.
Resource identifier	A piece of data that uniquely identifies a resource. For example, XPath can be used to uniquely identify elements inside an HTML document.
Stealthiness	The amount of human mimicking behaviours and JavaScript properties that gives the impression to websites that SAM is a human instead of a bot.
The dark web	The WWW content that exists on networks that use the Internet but require specific software, configurations, or authorisation to access [6].
Thread	One or multiple posts on a forum-like platform discussing a topic. A thread typically consists of a title and one or multiple posts by one or multiple authors posted on one or multiple dates.
Thread age	The time between the moment the web crawler started crawling and the moment the first post was posted by the author of the first post.
Thread length	The total number of posts in a thread.
Tor	The Tor (previously known as The Onion Router) network is a group of volunteer-operated servers that allows people to improve their privacy and security on the Internet. Tor achieves this by distributing user's transactions over several places on the Internet so no single point can link the user to his destination. Tor is free software and is open to everyone [7].
URL	URL, or Uniform Resource Locator, is a reference to a web resource that specifies its location on a computer network and a mechanism for retrieving it [8].
Workday	The time span of the day that the web crawler crawls, including the predefined breaks. An example time span would be from 8:00 to 17:00 CEST. This can be configured how the user wants, including the timezone.

1.3.2 ABBREVIATIONS AND ACRONYMS

Term	Description
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CEST	Central European Summer Time
CLI	Command Line Interface
ESA	European Space Agency
GUI	Graphical User Interface
IDE	Integrated Development Environment
HTML	HyperText Markup Language
LTS	Long Term Support
SAM	Stealthy Adaptable Machine
SDD	Software Design Document
SEP	Software Engineering Project
STD	Software Transfer Document
TLD	Top-level Domain
URD	User Requirements Document
URL	Uniform Resource Locator

1.4 LIST OF REFERENCES

- [1] Software engineering project group Crawling Dummies, "User Requirements Document," 2021.
- [2] ESA Board for Software Standardisation and Control (BSSC), "Esa software engineering standards," Standard ESA PSS-05-0 Issue 2, European Space Agency, Noordwijk, NL, 1991.
- [3] Rosie, "11 most popular types of captcha." <https://passwordprotectwp.com/types-captcha>.
- [4] "Overview of google crawlers (user agents)." <https://developers.google.com/search/docs/advanced/crawling/overview-google-crawlers>, Mar 2021.
- [5] "Cybercriminals." <https://www.trendmicro.com/vinfo/us/security/definition/cybercriminals>.
- [6] P. Biddle, P. England, M. Peinado, B. Willman, *et al.*, "The darknet and the future of content distribution," in *ACM Workshop on digital rights management*, vol. 6, p. 54, 2002.
- [7] "Tor: Overview." <https://2019.www.torproject.org/about/overview.html.en>, 2019.
- [8] Wikipedia contributors, "Url — Wikipedia, the free encyclopedia." <https://en.wikipedia.org/w/index.php?title=URL&oldid=1015459310>, 2021. [Online; accessed 29-April-2021].
- [9] M. Campobasso, P. Burda, and L. Allodi, "Caronte: Crawling adversarial resources over non-trusted, high-profile environments," *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2019.
- [10] Software engineering project group Crawling Dummies, "Software Transfer Document," 2021.
- [11] MongoDB, Inc., "Mongodb - the database for modern applications." <https://www.mongodb.com>, 2021.
- [12] MongoDB, "Mongodb compass." <https://www.mongodb.com/try/download/compass>. accessed on 28-06-2021.
- [13] "Docker." <https://www.docker.com>.

- [14] G. Acar, M. Juarez, and individual contributors, “tor-browser-selenium - tor browser automation with selenium.” <https://github.com/webfp/tor-browser-selenium>, 2020.
- [15] “Create React App.” <https://create-react-app.dev>. accessed on 07-06-2021.
- [16] “Electron.” <https://www.electronjs.org>. accessed on 07-06-2021.
- [17] M. Leotta, A. Stocco, F. Ricca, and P. Tonella, “Robula+: An algorithm for generating robust xpath locators for web testing,” *Journal of Software: Evolution and Process*, vol. 28, no. 3, pp. 177–204, 2016.
- [18] “flake8.” <https://flake8.pycqa.org/en/latest>. accessed on 09-06-2021.
- [19] “understand.” <https://www.scitools.com>. accessed on 09-06-2021.
- [20] “ewhm.” <https://github.com/parkouss/pyewmh>. accessed on 09-06-2021.
- [21] “pyautogui.” <https://github.com/asweigart/pyautogui>. accessed on 09-06-2021.
- [22] S. Behnel, M. Faassen, and I. Bicking, “lxml: Xml and html with python,” 2005.
- [23] L. Richardson, “Beautiful soup documentation,” *April*, 2007.
- [24] “pymongo.” <https://pypi.org/project/pymongo/>. accessed on 30-06-2021.
- [25] “tld.” <https://pypi.org/project/tld/>. accessed on 30-06-2021.
- [26] “websockets.” <https://pypi.org/project/websockets/>. accessed on 30-06-2021.
- [27] “flake8-polyfill.” <https://pypi.org/project/flake8-polyfill/>. accessed on 30-06-2021.
- [28] “pyflakes.” <https://pypi.org/project/pyflakes/>. accessed on 30-06-2021.
- [29] “pylint.” <https://pypi.org/project/pylint/>. accessed on 30-06-2021.
- [30] “pytest.” <https://github.com/HypothesisWorks/hypothesis-python>. accessed on 30-06-2021.
- [31] “pytest-cov.” <https://pypi.org/project/pytest-cov/>. accessed on 30-06-2021.
- [32] “pytest-datafiles.” <https://pypi.org/project/pytest-datafiles/>. accessed on 30-06-2021.
- [33] “xenon.” <https://pypi.org/project/xenon/>. accessed on 30-06-2021.
- [34] “pyyaml.” <https://pypi.org/project/PyYAML/>. accessed on 30-06-2021.
- [35] “python-dateutil.” <https://pypi.org/project/python-dateutil/>. accessed on 30-06-2021.
- [36] “termcolor.” <https://pypi.org/project/termcolor/>. accessed on 30-06-2021.
- [37] “pytz.” <https://pypi.org/project/pytz>. accessed on 30-06-2021.
- [38] “coverage.” <https://github.com/nedbat/coveragepy>. accessed on 30-06-2021.
- [39] “rich.” <https://github.com/willmcgugan/rich>. accessed on 30-06-2021.
- [40] “procpod.” <https://heptapod.host/saajns/procpod>. accessed on 25-06-2021.

1.5 OVERVIEW

The remainder of the Software Design Document consists of the following sections:

Section 2 provides an overview of SAM. This section addresses SAM’s relation to current projects, predecessors and successors. Moreover, it provides the function and purpose that SAM has. Also, it describes in which hardware environment and operating system SAM is developed and used. In addition, this section covers the users of SAM and the relation between SAM and other systems.

Subsequently, Section 3 contains an overview of the system architecture. This section describes the modular program structure, context viewpoints and the logical model of SAM. It also addresses SAM’s behaviour to certain events and how it acts in different states. On top of this, the model of the persistent data and the external interfaces are defined. Lastly, Section 3 concludes the rationale behind the system architecture.

Finally, Section 4 discusses the feasibility of SAM and the estimates for the resources needed to build, operate and maintain SAM.

2 SYSTEM OVERVIEW

This section addresses the relation of SAM to projects from the past, the present and the future. Moreover, the environment and purpose of SAM is described.

2.1 RELATION TO CURRENT PROJECTS

SAM has no relation to specific current projects.

2.2 RELATION TO PREDECESSOR AND SUCCESSOR PROJECTS

The predecessor of SAM is CARONTE [9]. Both SAM and CARONTE are web crawlers intended to crawl platforms on the dark web and gather information without being noticed. Both are used for the research project of the client. CARONTE was created by the client and is stealthy, but CARONTE is neither robust nor scalable. For a web crawler to keep gathering information and work on multiple platforms, the web crawler must be robust. Hence, SAM improves CARONTE by being more robust.

No successor projects are planned to our knowledge.

2.3 FUNCTION AND PURPOSE

The main purpose of SAM is to crawl platforms on the dark web and gather information written on these platforms. This purpose can roughly be divided into the following four tasks.

- User interaction and configuration;
- Learning the platform structure;
- Gathering data;
- Navigating a platform.

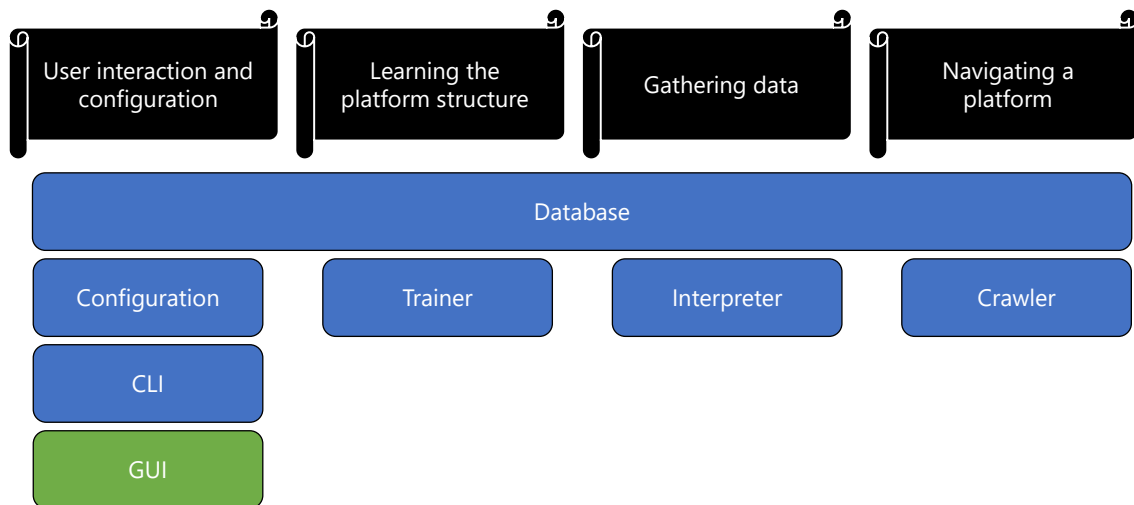


Figure 1: The tasks of SAM and the corresponding modules that handle them. Tasks are shown on black scrolls; modules that handle the task are depicted as rectangles below them.

SAM is divided into seven components (six Python modules and the GUI); each component is designed to handle one or more of the above tasks. The seven components, including brief explanations of their functionality, are listed below.

- Crawler: navigating platforms.
- Database: communicating with the MongoDB database.
- Interpreter: understanding the web pages that are crawled and extracting data from them.
- Trainer: learning the platform structure.
- Configuration: providing the data structure for the crawler configuration.
- CLI: providing an interface for interacting with SAM during the crawling process.
- GUI: providing an interface for starting, configuring, and training SAM.

Figure 1 gives an overview of the four tasks and the corresponding modules that handle them. How exactly the modules contribute to solving the task will be explained in Section 3.

2.4 ENVIRONMENT

A diagram is given in Figure 2 which summarises the environment.

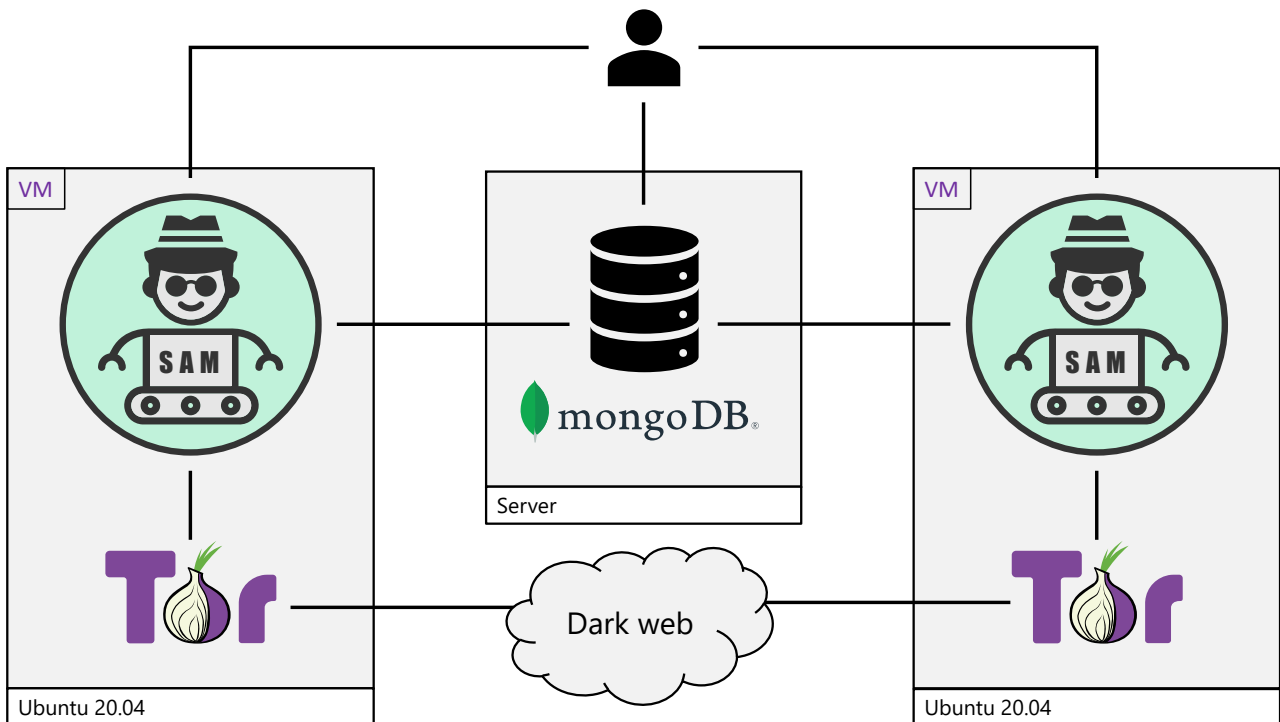


Figure 2: Example environment with two instances of SAM running. The text 'VM' indicates that it is recommended for the two instances of SAM to run in a virtual machine (VM).

Two machines are needed to make SAM function. One machine, running the Ubuntu 20.04 operating system, runs an instance of SAM. This instance of SAM controls a Tor Browser 10.0.16 process for accessing the dark web. The other machine runs MongoDB 4.4.6, providing the database that SAM communicates with. It may be desirable to run multiple instances of SAM; in this case, multiple (virtual) machines are needed, each running Ubuntu 20.04. Each machine can run one instance of SAM. SAM has one user, which is the client, interested in gathering information from platforms. This user directs the instance(s) of SAM, and accesses the database. Note that it is not required for the database to run on a machine separate from the SAM instances.

While developing SAM, we used continuous integration (CI), which is discussed in more detail in the Software Transfer Document (STD) [10]. The CI was run on a separate machine with the Ubuntu Server 18.04 operating system. It was run inside a 'Docker container' using Docker Engine 20.10. During development, the database was run on the same machine.

Next, we will elaborate on each part of the environment.

2.4.1 OPERATING SYSTEM

SAM runs on Linux. For this project, we have only run SAM on Ubuntu 20.04. Therefore, this is the only version for which it is certain that SAM works as intended. The machine containing the database and the CI uses Ubuntu Server 18.04. This version was most accessible to the team due to the university providing Ubuntu Server 18.04 as the standard distribution for virtual machines.

2.4.2 MONGODB

MongoDB [11] is a NoSQL database. An easy way one can view data present in a MongoDB database is the MongoDB Compass [12] application. This application provides a GUI for viewing the data present in any database.

2.4.3 DOCKER

Docker [13] is an open platform for developing, shipping, and running applications. With Docker, applications are packaged into containers. These containers contain everything needed to run the application. Thus, these containers work the same on every device that allows the container to run. When running multiple of these containers, they all run on the same operating system kernel, meaning that they use fewer resources than virtual machines. Docker containers are great for continuous integration, because everyone can run the test containers, and they all run in the same way. This makes sure that all tests are executed like they are supposed to.

2.4.4 TOR

Tor [7], previously known as The Onion Router, is a software package that implements onion routing. Onion routing is the process of encrypting connections multiple times and forwarding them along with a number of nodes in such a way that no single node knows the source and the destination of the packet at the same time. In this way, Tor provides anonymous browsing of web pages. Besides allowing anonymity, Tor also provides functionality for hosting onion services on the Tor network. These onion services are websites that have a *.onion top-level domain (TLD) and can only be accessed through the Tor network.

2.4.5 THE DARK WEB

The dark web is a part of the World Wide Web that can only be accessed with specific software, configurations, or authorisation. For example, the onion services mentioned in Section 2.4.4 are part of the dark web. Due to its anonymity, the dark web is often used for illegal and illicit practices like child pornography and drug markets. Our crawler will access the dark web, more specifically onion services, because the client is specifically researching illicit goods on dark web communities.

2.4.6 USER

SAM is a custom-built research tool for the client, and therefore, the client is the sole user. SAM runs mostly autonomously to gather data from different platforms. The user does not need to continuously interact with the crawler and will take on a passive role after the initiation and training of the crawler instances. The user can access data gathered by the crawling in the database; this can be done using, for example, MongoDB Compass.

2.5 RELATION TO OTHER SYSTEMS

SAM is an independent system made to replace CARONTE. SAM uses two larger systems: The Tor browser and the MongoDB database. The Tor browser is autonomously controlled with tbselenium [14], a package made for Python to interact directly with the Tor browser from Python code. Secondly, SAM interacts with the MongoDB database using the query language specified by MongoDB. This database is used to store data collected by the crawler process and to store and retrieve platform structures that the crawler has learned.

3 SYSTEM ARCHITECTURE

This section addresses the architectural design of SAM. Specifically, the decomposition of the system is provided in detail and for each module or component, the most important functionality is explained.

3.1 ARCHITECTURAL DESIGN

The architectural design of SAM follows a 3-tier application architecture, which is a client-server architecture where presentation, application processing and data management functions are separated in three layers as shown in Figure 3. The presentation layer is the front-end of SAM, including the GUI and the CLI. The GUI provides the user with an interface to start, configure and interact with SAM. The CLI displays information about SAM and allows the user to change the state of SAM while it is running. The back-end of SAM consists of the application layer and the data management layer, which handle the actual crawling with all relevant data processing and sending this data to the database.

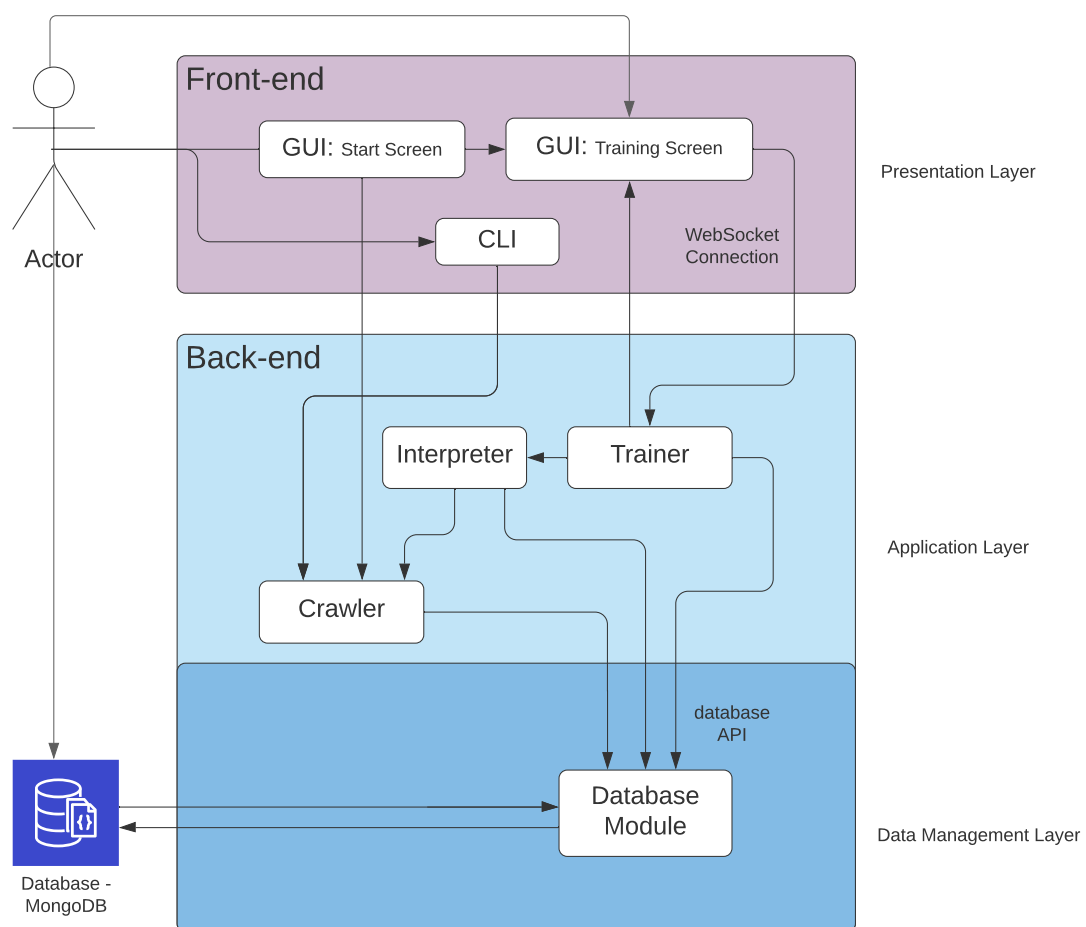


Figure 3: Global system architecture of SAM.

3.1.1 FRONT-END

The front-end of SAM manages the interaction with the user. The front-end is responsible for presenting the user with two separate GUIs, which are the start screen interface and the training screen interface, and one CLI.

When the user launches SAM, the system displays the start screen interface, which serves as a screen where the user configures and starts the crawling process of SAM. Once the user inputs the configuration parameters, and this configuration is submitted successfully, SAM starts crawling. When the crawler starts, it will start a training phase. Even if SAM already knows the platform, SAM still enters a training phase. During this training phase, the training screen interface appears multiple times. The training screen allows the user to indicate to SAM what different HTML elements represent and to confirm that the stored platform structure is accurate. If the structure is not accurate, SAM requests the user to adjust it. These elements are then sent to the back-end via a WebSocket connection, such that the trainer component of the system can perform training.

While crawling, the CLI acts as an interaction point between the user and the crawler. The user can enter commands into the CLI to pause, start and terminate the crawler. Meanwhile, the crawler logs information, such as the current page type, fetch delay, start and end time of the workday, breaks and interruptions, to the CLI.

3.1.1.1 GUI module

The GUI is an application created using the Create React App [15]. In addition, Electron [16] is used to create a standalone desktop application with JavaScript, HTML, and CSS.

As stated before, There are two main views in the GUI: the start screen interface and the training screen interface. The user first interacts with SAM using the start screen interface. During the training phase, the user interacts with SAM using the training screen interface.

Start screen interface

The start screen interface, shown in Figure 4a, is the first screen that the user can interact with. It provides the user the possibility to instruct SAM which platform to visit and change all the available predefined input values that are used during crawling. These can be left empty if the user does not want to change the predefined values. Furthermore, the user has the option to input relevant keywords and blacklisted keywords; the keyword screen is depicted in Figure 4b. Lastly, the start screen interface provides the user with a weekly schedule, as shown in Figure 4c, that SAM follows when an instance has started. A schedule consists of workday(s) and break(s). Once the user is done adjusting the values to his own preference, this configuration is stored and sent to the crawler. The crawler then acts according to the created configuration.

Training screen interface

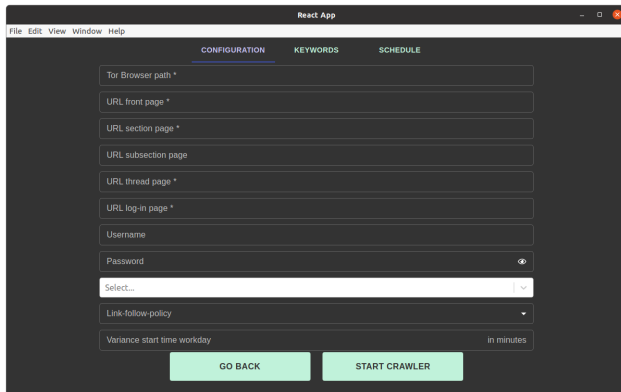
The training screen interface, shown in Figure 5a, is the screen that the system displays when the user starts a new instance of SAM.

The training screen interface provides the user with the option to label the structure of the platform of a specific page type. Once the user has finished labelling a page, the Trainer module trains this web page and sends it back to the training screen interface. The user can then confirm this structure or adjust it if needed, as shown in Figure 5b. The next time an instance of SAM is launched, if the platform has already been trained, the training screen interface shows the previous labelling, such that the user can immediately train the structure. We further discuss the process of training in Section 3.2.3.

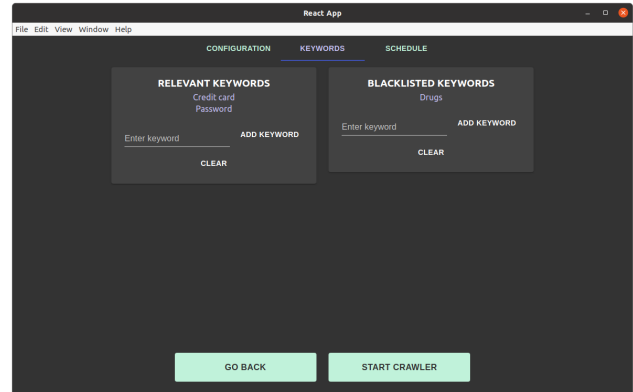
3.1.1.2 CLI module

The design of the CLI module aims to extend the functionality of the user's interaction with SAM during the crawling session. The CLI shows the process of SAM and gives the user the option to interact with SAM while the crawling is ongoing. The CLI shows detailed information of what SAM does, and allows the user to enter commands to pause, start and terminate the crawling.

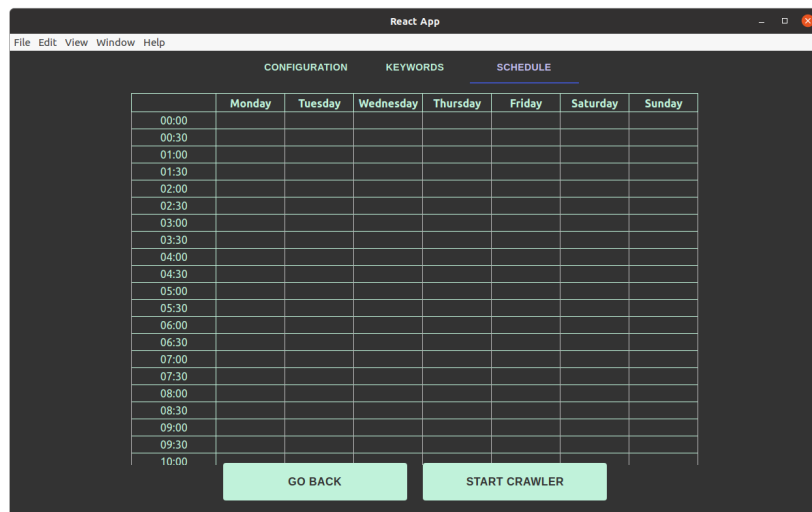
We decided to use a CLI for interaction with the crawler while it is crawling instead of a GUI, because the crawler runs mostly autonomous. Little to no interaction from the user is expected while the crawler is running. Thus, creating a GUI for interaction with the crawler during run time would not be worth the time it takes to create it.



(a) The screen to input some configuration values.

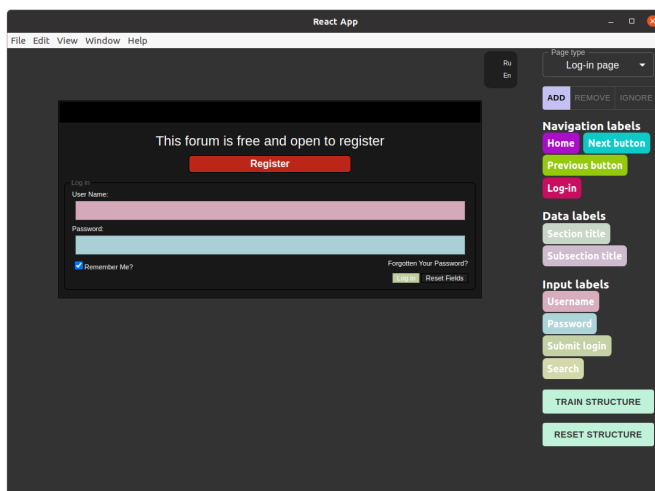


(b) The screen to input the keywords for crawling.

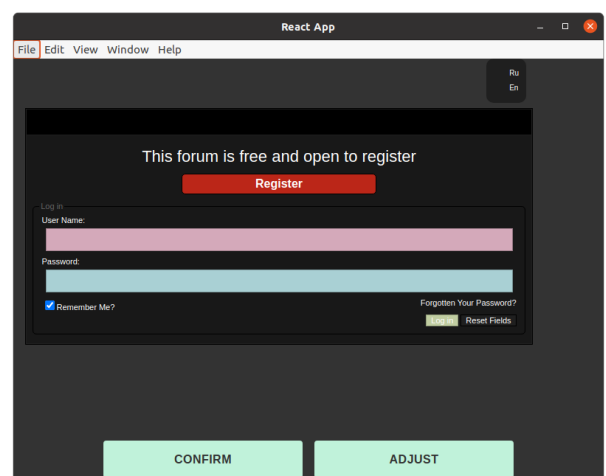


(c) The screen to input the schedule for crawling

Figure 4: The start screen interface.



(a) The main screen for the training session to label a web page.



(b) The double check screen after training.

Figure 5: The training screen interface.

3.1.2 BACK-END

The back-end of SAM contains the server-side and utility side of the application, as well as the communication to the database. The back-end design of SAM follows parts of the SOLID design principles:

- **Single Responsibility Principle:** A class has one job to do. Each change in the requirements can be done by changing just one class. SAM applies the principle by separating functionalities of SAM into different modules.
- **Open Closed Principle:** Software entities, such as classes, modules, and functions, should be open for extension but closed for modification. SAM applies the principle by using `get_something` methods in a class when another class is trying to access the attributes of it.
- **Liskov Substitution Principle:** Objects of a superclass shall be replaceable with objects of its subclasses without breaking the application. This principle is the extension of the Open Closed Principle. SAM does not apply the principle, since the situation is not applicable for SAM.
- **Interface Segregation Principle:** No client should be forced to depend on methods it does not use. SAM applies this principle by letting modules communicate with each other through APIs, such that different modules are loosely coupled.
- **Dependency Inversion Principle:** High-level modules should not depend on low-level modules; both should depend on abstractions. Moreover, abstractions should not depend on details. Instead, details should depend on abstractions. SAM applies the principle in the trainer module when there are three methods to identify the structure of a page. An abstract class is defined, and three concrete classes inherit from it, with each class implementing one method.

3.1.2.1 Server side

The server side of SAM has three modules: the Crawler module, Interpreter module and Trainer module. The different modules aim to separate the functionality of SAM according to the divide and conquer strategy. The sequence diagram in Figure 6 shows how the different modules in the back-end work together with the front-end before the crawling process starts.

We will briefly explain Figure 6. When the user starts SAM, the GUI will be started first. After the user enters the desired configuration, the GUI will run a Python script to start the back-end. This script creates an instance of the crawler, which in turn initialises the interpreter. Subsequently, the interpreter creates the trainer. In addition, the CLI is started, but this is left out of the diagram, since it is not used before the crawling process. After the crawler fetches its first web page, and stores the assets in the database, it will call the interpreter to parse the web page. However, before the interpreter does this, it will start a training session. This training session involves communication between the trainer, GUI and database modules.

Next, we elaborate on the three server-side modules.

3.1.2.1.1 Crawler module

The crawler module is the core part of the back-end. We designed it in such a way that it only has functionalities that are related to the crawling process. These functions are navigating the web pages, checking the schedule and processing a web page. The crawler module has a strong connection with the interpreter module, sending web pages to it and receiving a parsed web page back. This parsed data is then used to retrieve information and locations of certain web elements for navigating and browsing the web page.

3.1.2.1.2 Interpreter module

The interpreter module is designed to process the retrieved web pages from the crawler module. It uses the platform structure retrieved from the trainer to find the relevant information and extract it. The interpreter is also responsible for determining the type of page being processed, determining whether more training is needed or not and deciding whether a thread is already parsed, contains new information or is not yet parsed. The extracted data is then stored in the database and some information is reported back to the crawler, such that it has all the information it needs to make a decision on what to do next.

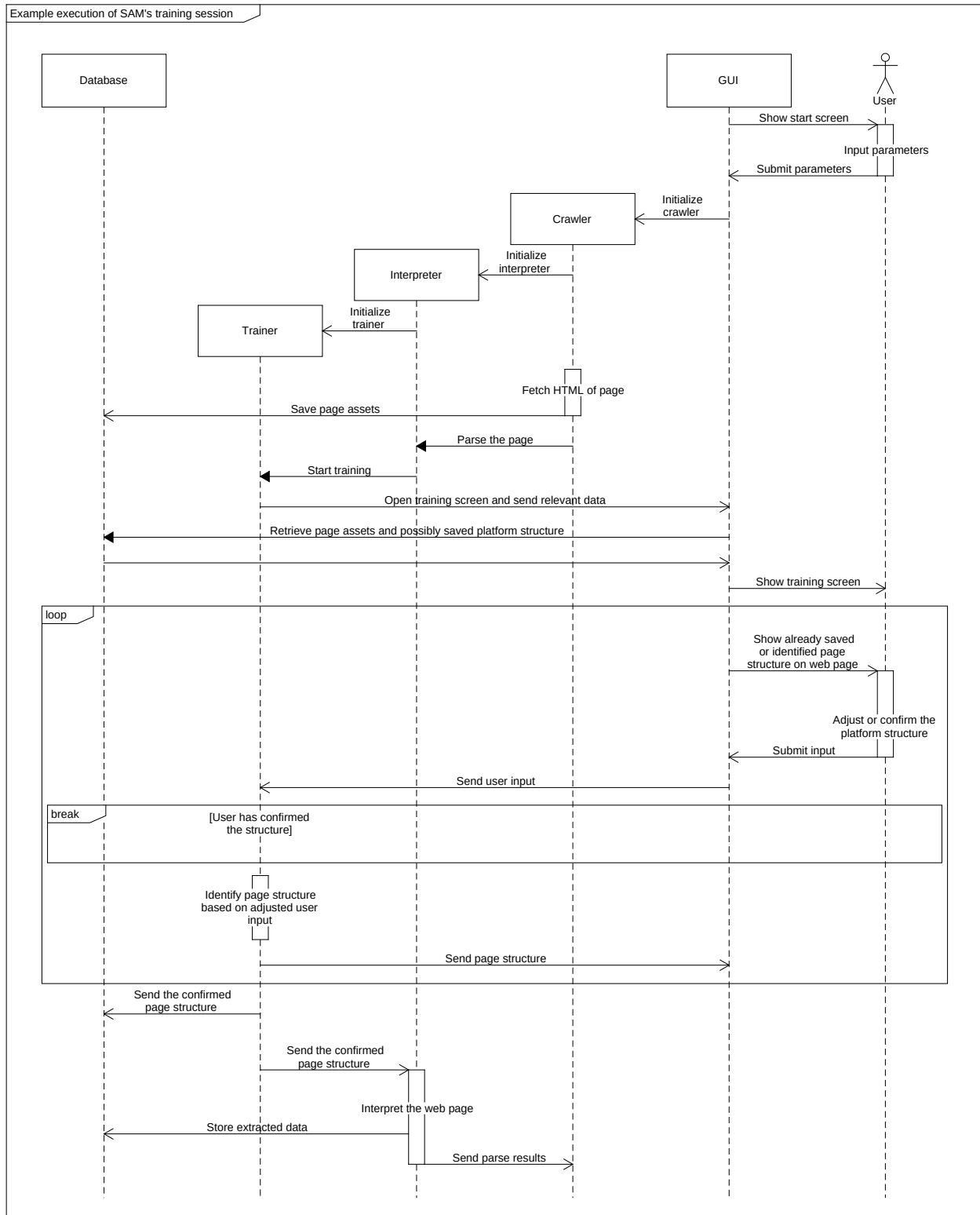


Figure 6: Sequence diagram of SAM's training session showing the interaction between components.

3.1.2.1.3 Trainer module

The trainer module handles the process of identifying the structure of a platform. This identification is performed based on user input received from the GUI. The workings of the trainer module are elaborated on in Section 3.2.3.

3.1.2.2 Utility side

A utility class is a class that contains commonly used functions. SAM has three modules containing utility classes: the enum module, configuration module and the database module.

3.1.2.2.1 Enumeration

During the development of the back-end modules, we found that some enumerations were defined in more than one place. This is because multiple modules use the same enumeration values. To prevent this, we decided to create a new module containing the definitions of all enumerations: the enum module. If a module uses an enumeration, it should then be imported from the enum module instead of re-defining it. The following enumerations are retrieved from the enum module:

- **CAPTCHA Type.** This enumeration contains different CAPTCHA types. This class is used in the CAPTCHA-solving method in the crawler module and in the CAPTCHA detection part of the interpreter module.
- **Structural Element.** This enumeration contains the types of elements on a web page that are of interest. There are three subclasses inherited from it. First, the `DataElement` class contains the types of data-carrying elements on a web page. Next, the `NavigationalElement` class defines the types of navigational elements on a web page. Lastly, the `InputElement` class holds the types of input elements on a web page. The class `StructuralElement` is used in the trainer module, interpreter module and crawler module.
- **Page Type.** This enumeration contains the different types of pages SAM is interested in. This class is used in the trainer module, interpreter module and crawler module.
- **Relevance.** This enumeration contains the different levels of relevance throughout the project. This class is used in the interpreter module and crawler module.
- **Thread Status.** This enumeration contains the status of a thread. The status determines whether it has been crawled before, is not parsed yet or has new posts and needs to be recrawled. This is used in the interpreter module.

3.1.2.2.2 Configuration

The configuration module is designed to store the configuration of SAM and allow other modules to retrieve the information about the configuration during the crawling process. SAM has predefined values, which are used if the user does not provide any input values. These input values can be submitted in the training screen interface. Whenever the configuration of the system is changed, the configuration module saves the current configuration to the database.

3.1.3 DATABASE MODULE

Since SAM aims to obtain all relevant web page contents and save internal configurations, the retrieval and storage of data are essential for the whole system. The database module in the back-end provides all functionalities that are related to communication with the database. These functionalities are provided through a single class instance. Because the database library (PyMongo) can sometimes be wearisome to use, the database module defines a custom API on top of the library, making it much more user-friendly. The custom API does not expose all the functionality of the library. Therefore, the library's API is made accessible as well, such that it can be used in cases where functionality in the custom API is not sufficient.

3.2 LOGICAL MODEL DESCRIPTION

Having described the architectural design, we will now describe the logical model of SAM. First, we discuss the role of each module and the connection between the modules. Subsequently, we address the modules more thoroughly by describing the functionality of each of the classes in a module, as well as the connections between these classes. Figure 7 shows the complete class diagram of SAM and visualises these modules and their connections.

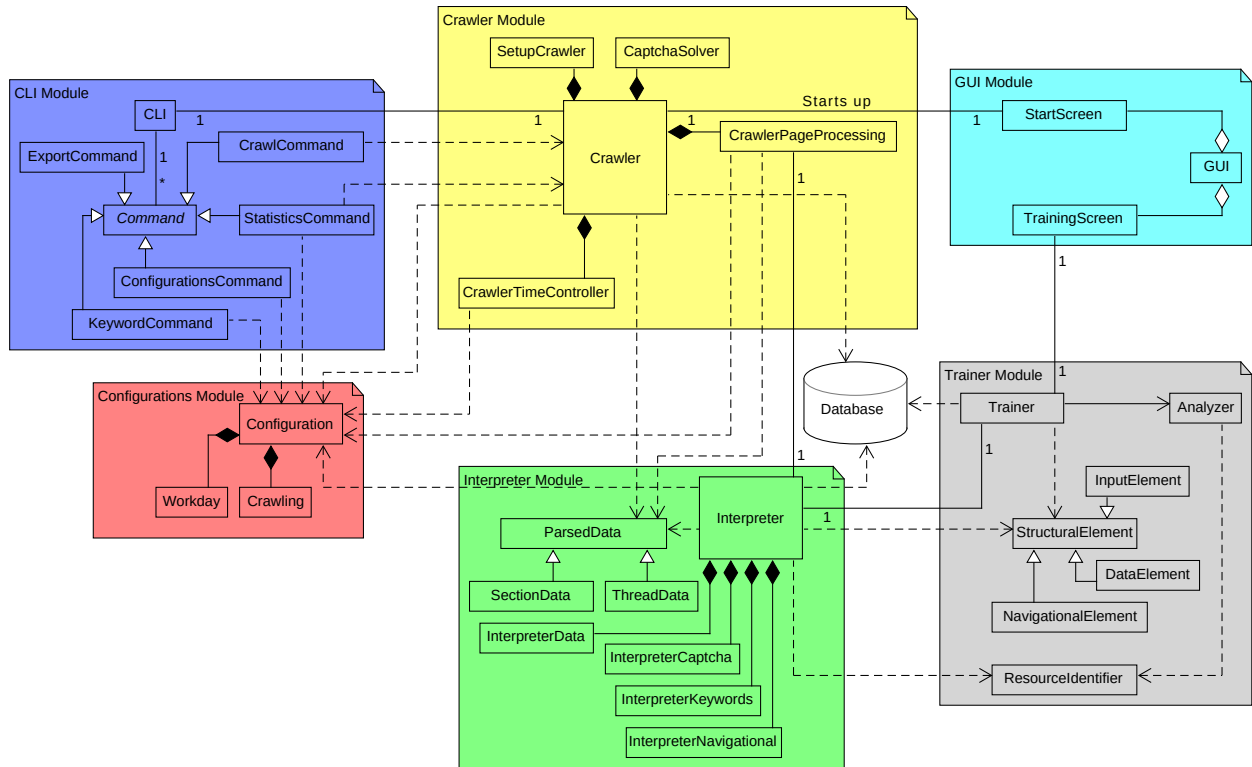


Figure 7: Class diagram of SAM's structure.

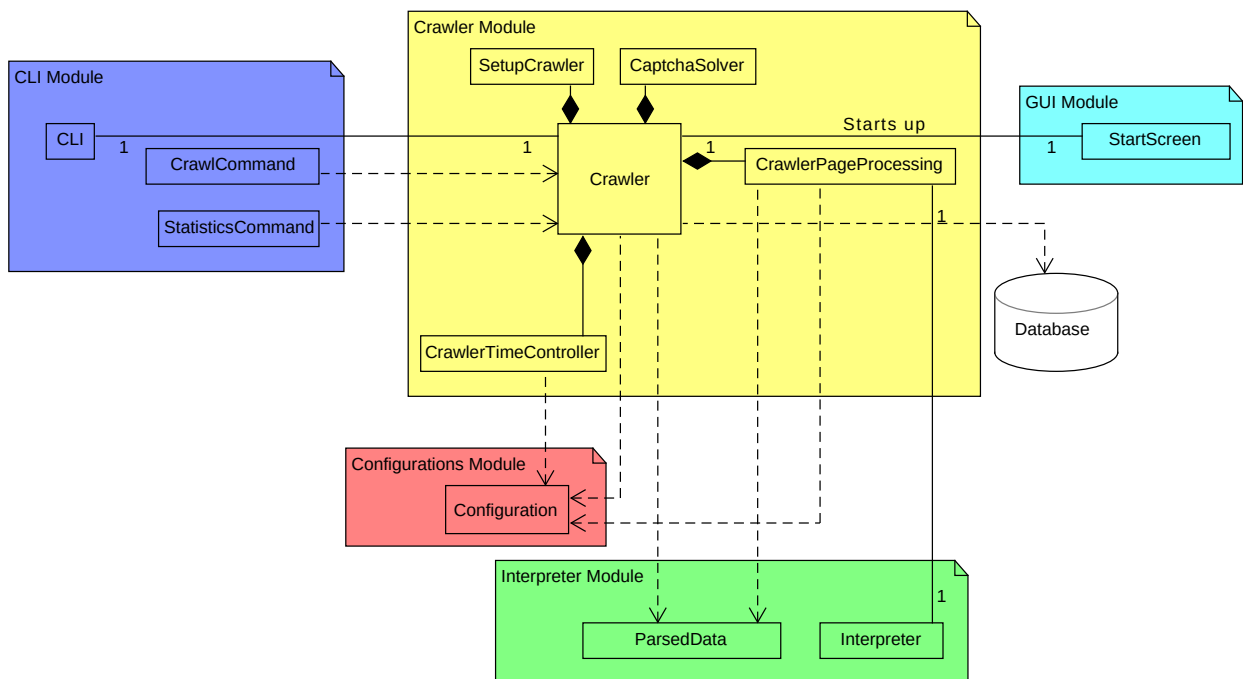


Figure 8: Class diagram of SAM's structure, zoomed in on the crawler module.

3.2.1 CRAWLER MODULE

The crawler module is the main module of SAM. This module determines which pages SAM visits and when SAM should take a break. This module also calculates the amount of time SAM should stay on each page. The crawler module consists of five classes, as can be seen in Figure 8 (in yellow).

The *Crawler* class is the main class of this module. It holds instances of all other classes in this module, which *Crawler* uses to make decisions. This class handles stopping, pausing and terminating the crawler. It also makes decisions on the crawling process based on the information it gathers from the other modules. In general, this class does not directly interact with the web page, but it controls the *PageProcessing* class that interacts with the web pages.

- *SetupCrawler* class: The *Crawler* class uses this class to get everything ready such that the crawler can start running. After the crawler terminates, this class will make sure to close everything accordingly. One concrete example of what this class does is start the Selenium webdriver and launch a Tor Browser for the main *Crawler* class to use when the *SetupCrawler* class is created.
- *TimeController* class: This class creates and stores the schedule of the crawling session. The main *Crawler* class uses this class to determine when to pause and how long the crawler has to remain paused. At the beginning of each session the *Crawler* class creates this class and a schedule is made for the rest of the day. This class will check at every page whether there is a break, and if there is one, how long it will take. Another functionality of this class is to add breaks or interrupts during the crawling session.
- *CaptchaSolver* class: This class notifies the user that a CAPTCHA is found if SAM identifies a CAPTCHA on a web page. The user can then complete the CAPTCHA such that the crawler can continue crawling. Since the CAPTCHA will be solved by the user, this class waits for input of the user such that it can continue crawling the pages.
- *PageProcessing* class: This class fetches the next page to crawl and initialises the crawler using the configuration module. Furthermore, the class sends the pages to the interpreter and relays the results from the interpreter back to the *Crawler* class. The main *Crawler* class calls the `next_page` function of this class repeatedly to process all of the pages. The *PageProcessing* class looks at the current page during each call. It will determine the type of the page with the help of the interpreter module, and it can make a decision about what to do and where to go next based on this type. Because the *PageProcessing* class has to make these decision, the class will also keep track of visited elements, such that no thread is visited twice. Additionally, the *PageProcessing* class saves the page to the database when retrieving the page, and it translates the navigational elements from the interpreter to elements that the Selenium driver can find.

3.2.2 INTERPRETER MODULE

The interpreter module is responsible for extracting information from the web pages the crawler retrieves. The interpreter is built out of one main interpreter and four sub-interpreters. Furthermore, the interpreter module contains *ParsedData* classes, which contain relevant data to send to the crawler. The exact contents of this data are further explained alongside *ParsedData*. The structure of the interpreter module can be seen in Figure 9 (in green).

The interpreter functions by using several sub-interpreter classes. The sub-interpreters all execute their task, as shown in Figure 9:

- The main interpreter: This class connects the interpreter module to the database, the trainer and the crawler. It prepares the data for the sub-interpreters, passes the data from sub-interpreters to the *ParsedData* class and determines the type of page SAM has crawled to. If the platform structure is unknown, the interpreter starts the trainer for a training session.
- The *InterpreterData* class: The first sub-interpreter that the main interpreter calls. This class extracts information from the web page and saves this information in *ParsedData*. Extracted data includes, but is not limited to, the authors' usernames, the date a post was made and the thread age. The *InterpreterData* class also counts the number of words on a thread page in order to help the crawler determine how long it should wait until fetching a new page.
- The *InterpreterNavigational* class: This class locates the navigational elements on a web page. Examples of navigational elements are the buttons to go to the next page or the buttons to go to the previous page. Furthermore, if these elements cannot be found, this class determines that the page is badly formatted and cannot be parsed correctly.

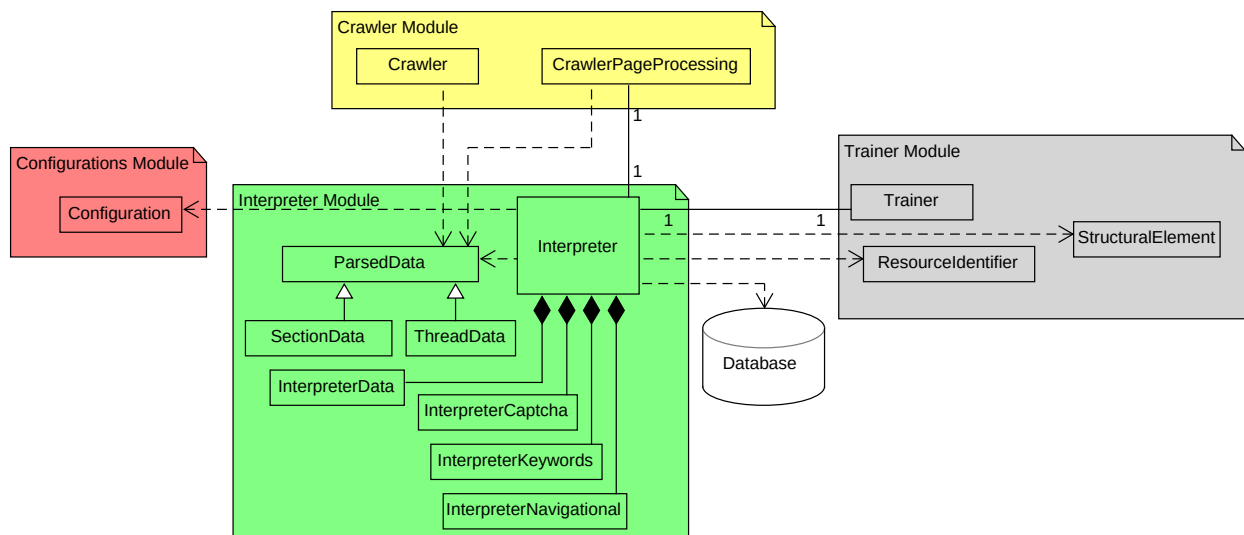


Figure 9: Class diagram of SAM's structure, zoomed in on the interpreter module.

- The `InterpreterCaptcha` class: This class searches for CAPTCHAs that may appear on the web page. If this interpreter detects a CAPTCHA, it will also determine the type of CAPTCHA. However, this class cannot detect every type of CAPTCHA, especially if the CAPTCHA is rarely used or uniquely designed. The `InterpreterCaptcha` class can detect CAPTCHAs of the types Simple Math, Word Problem, reCAPTCHA, noCAPTCHA reCAPTCHA, invisible reCAPTCHA and Honeypot.
- The `InterpreterKeywords` class: This interpreter class determines the relevancy of the web page based on the relevant and blacklisted keywords, as well as the link follow policy. `InterpreterKeywords` distinguishes between elements to use to determine the relevancy based on the page type. For instance, if the page type is a thread page, then the post contents and the thread title will be used. If the page type is not a thread page, the thread titles or section titles are used to determine the relevancy for each thread or title.

The information that the interpreter module extracts is passed along to the crawler module using the `ParsedData` class and its subclasses: `ThreadData` and `SectionData`.

- The `ParsedData` class: This class contains information that is relevant for the crawler or the database. The type of page determines which subclass is used. `ParsedData` holds basic information that is relevant for all page types. This includes the following fields:
 - Next page button
 - Previous page button
 - Section button
 - Home button
 - Login button
 - Button to go to the login page
 - Button to go to the first page of a thread
 - Is a CAPTCHA on the page
 - CAPTCHA type
 - CAPTCHA location
 - Username field
 - Password field
 - Is the page badly formatted

The only field that has some more implications for the crawler is whether the page is badly formatted or not. If the page is badly formatted, the CLI informs the user that a badly formatted page is encountered. In addition, the crawler module then knows that all of the information in `ParsedData` may be non-existent, even if it is expected to exist. This mark is also applied in the database to indicate that the information on this page may need to be manually extracted. If the page is not badly formatted, the crawler may assume that all information that should be present on the page is present and found.

- The `ThreadData` class: This class is used to relay information found specifically on a thread page. It contains all the information that the `ParsedData` class holds, as well as the following additional information:
 - Page relevancy
 - Number of posts in the current thread
 - Thread age
 - Is the thread parsed completely
- The `SectionData` class: This class is used to pass information found specifically non-thread pages to the crawler. It contains all the information that the `ParsedData` class holds, as well as the following additional information:
 - Thread list
 - Thread status
 - Thread relevancy
 - Section list
 - Subsection list

3.2.3 TRAINER MODULE

The trainer module is responsible for training SAM to understand the platform structure. The structure of the trainer module can be seen in Figure 10 (in grey).

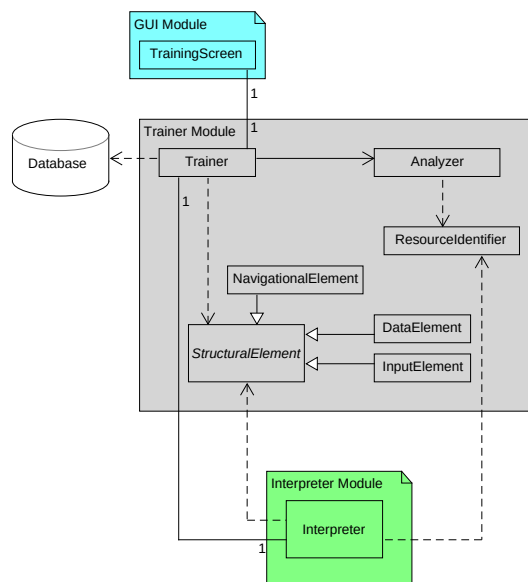


Figure 10: Class diagram of SAM's structure, zoomed in on the trainer module

The `Trainer` class forms the backbone of the training process. First, the `Trainer` class contains the method that the interpreter module calls to initiate training. The goal of training is for SAM to learn the structure of different web pages. An important part of learning is remembering. Therefore, the `Trainer` class also has a connection to the database for retrieving previously learned platform structures and storing newly learned ones. Furthermore, training is done based on user input entered in the training screen of the GUI. Hence, there is a connection between the trainer and the training screen.

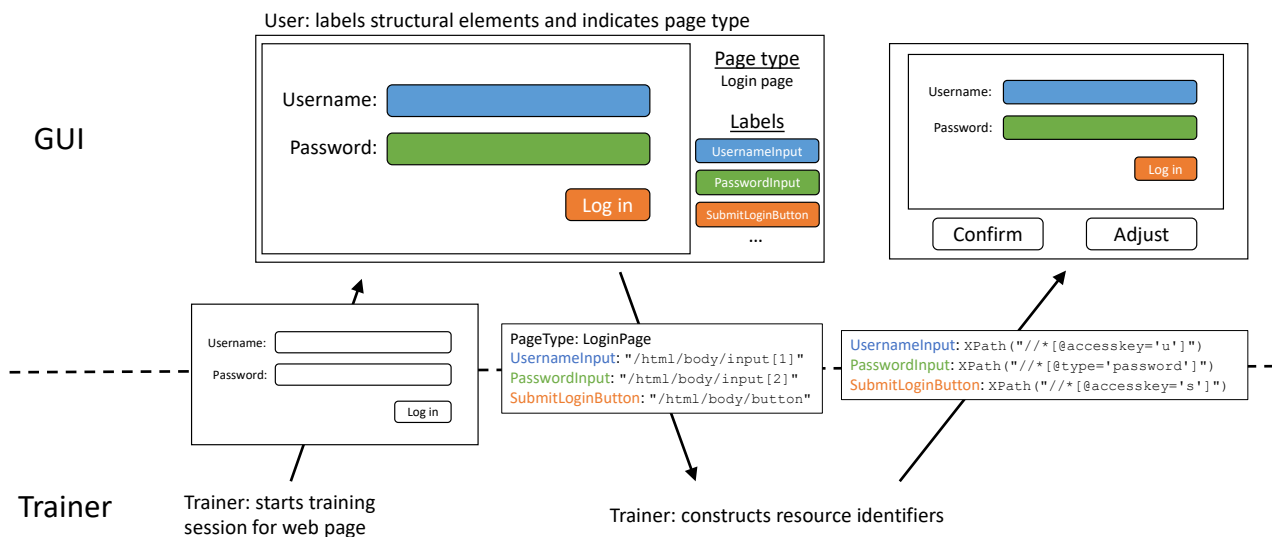


Figure 11: Diagram of basic training process.

In order to argue the existence of the other trainer classes, we first need to explain how the training works. A diagram of the basic training process is given in Figure 11. At the start of a training session, the trainer sends information of the page that needs to be trained to the GUI. In the training screen, the user indicates the type of page and labels HTML elements on the web page. In Figure 11, a log-in page is given. Therefore, the user indicates that it is a log-in page, and will give the labels `LoginButton`, `UsernameInput` and `PasswordInput` to the appropriate HTML elements. The GUI then sends the user input to the trainer. HTML elements highlighted by the user are represented by XPath expressions. XPath (XML Path Language) is a query language to select nodes in an XML document, but it can also be used for HTML documents. After receiving the user input from the GUI, the labels are represented by a `StructuralElement`. We distinguish three types of structural elements on a web page:

- `DataElements` contain information that the interpreter will save to the database;
- `NavigationalElements` are used by the crawler to navigate through the forum;
- `InputElements` are used by the crawler to enter values into HTML elements.

The type of page is stored in the database without further involvement of the trainer, but the selected HTML elements are first analysed. Based on the selected HTML elements, the intended structural elements are identified on a web page. To abstract away from this process, the `ResourceIdentifier` and `Analyzer` classes are present. A `ResourceIdentifier` provides some way of identifying an element, and an instance of a class that inherits from `Analyzer` is able to compute such a `ResourceIdentifier` for a certain `StructuralElement` based on the user-selected elements.

In Figure 11, a simple example is given where, for instance, the structural element `PasswordInput` is identified by the XPath expression `//*[@type='password']`. In this case, a direct path to the element (like `/html/body/input[2]`) would also suffice, but would be less robust. The identification process becomes more challenging when identifying elements such as thread titles, since there can be many thread titles on a web page, whereas there is only one password input field.

3.2.3.1 Training methods

The trainer module uses two methods for constructing identifiers. One is an adapted version of the Robula+ algorithm [17], which is an algorithm for generating robust XPath expressions for uniquely identifying an element. Very roughly, the Robula+ algorithm works as follows, starting with the most general XPath expression (`//*`):

1. Does the XPath uniquely identify the target element on the web page?
 - (a) Yes? Stop, and return the XPath.
 - (b) No? Add information about the target element to the XPath, and go to 1.

The idea is that the above process results in the most general XPath that still uniquely identifies the target element. One downside of the algorithm is its high running time complexity. The paper states that it is “exponential in the number of predicates (i.e., attributes, text and position) and levels that can be found between the target element and the root of the DOM.”

Our second method, which is mainly used for data elements like thread titles, finds the common path of elements and uses that for identification. For instance, assume the user highlights the following elements:

Then the method outputs the XPath `/html/body/div/a[3]`. For both methods, the trainer also has support for marking elements that should not be identified by a resource identifier.

3.2.4 CONFIGURATION MODULE

The configuration module stores the configurations for a crawling session. The configurations are set by the crawler and can be changed or shown through the CLI. The structure of the configuration module can be seen in Figure 12 (in red).

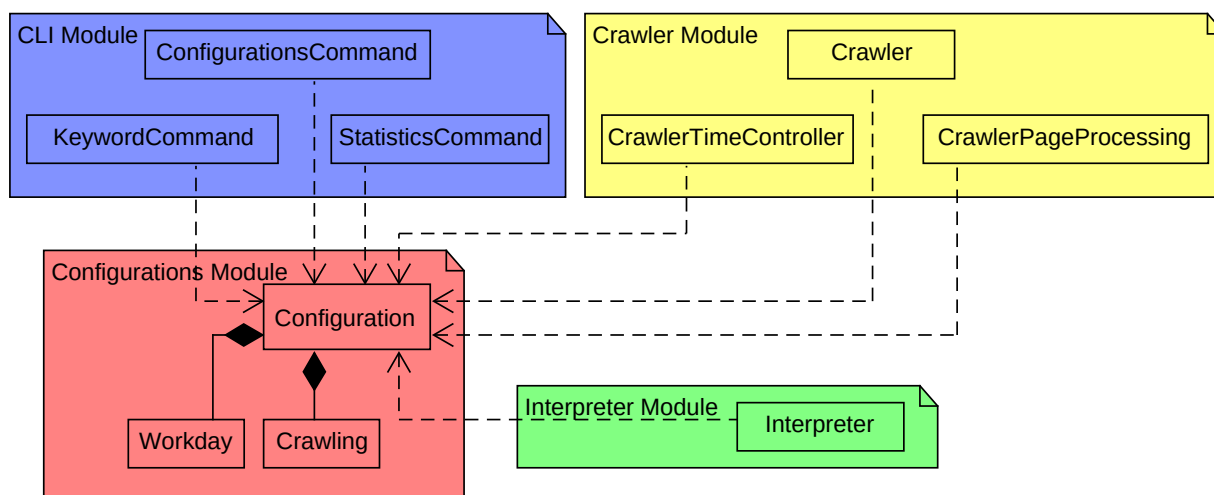


Figure 12: Class diagram of SAM's structure, zoomed in on the configuration module.

The `Configuration` class contains a reference to one instance of the `Workday` and one instance of the `Crawling` classes. The `Workday` class stores the configuration of the workday, which includes the start and end time of the workday, start and end times of breaks and the timezone. The `Crawling` class stores other configurations, namely the list of blacklisted keywords, the maximum thread length and the link follow policy. The `Configuration` class can also save the configuration.

3.2.5 GUI MODULE

The GUI module consists of two screens that the user can interact with: the start screen and the training screen. The structure of the GUI module can be seen in Figure 13 (in cyan).

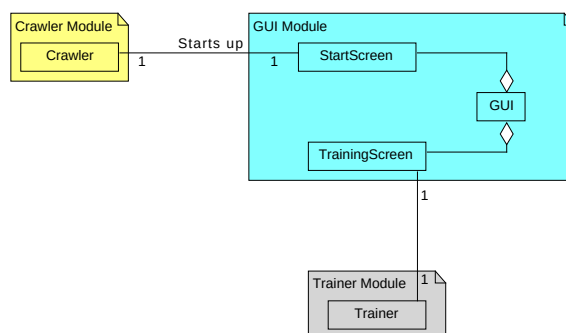


Figure 13: Class diagram of SAM's structure, zoomed in on the GUI module.

The `StartScreen` class contains all the components for inputting the configuration. This configuration is stored and sent to the crawler once the crawler is started. The start configuration contains all the values needed for the configuration of the crawling instance. To allow the use of default values, the fields can be left empty as well.

The `TrainingScreen` handles all user input for training the structure of the platform. This screen will show the web page as sent by the trainer along with labels that allow categorisation of the selected elements. There are three utility options: Add, Remove, Ignore. Depending on which option is selected, the result of clicking on an element in the web page differs. For the Add option, clicking an element highlights it with the selected label. Remove removes all highlighting and removes the element from the stored list. Lastly, Ignore will highlight the element using a grey colour and adds the element to the ignored elements. Finally, there is also the option to specify a date format for each label. When a structure is trained, it will be shown in a separate window to ensure the structure is correct. If it is not correct, the structure can be adjusted in the `TrainingScreen`.

3.2.6 CLI MODULE

The CLI module consists of one CLI that the user can interact with and several types of commands that the user can call. The structure of the CLI module can be seen in Figure 14 (in blue).

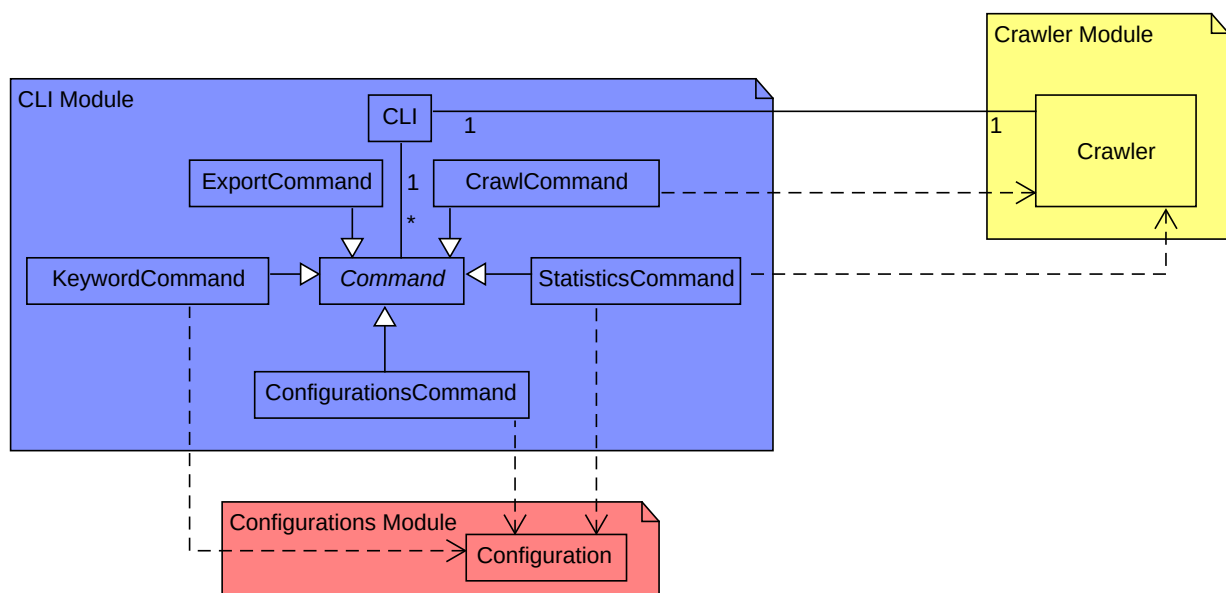


Figure 14: Class diagram of SAM's structure, zoomed in on the CLI module.

The CLI will notify the user when a page is badly formatted and allows the user to type commands. These commands, of which there are five types, are then executed by the system. The commands are listed and explained below.

- `KeywordCommand`. This command handles the addition and removal of relevant and blacklisted keywords. The user can also call a command to show the list of relevant or blacklisted keywords.
- `ConfigurationsCommand`. This command type allows the user to change certain configurations, such as the maximum reading speed or the maximum thread age. These configurations can also be shown.
- `StatisticsCommand`. This command type asks the CLI to show crawling statistics. Statistics include the end time of the workday, the current platform SAM is crawling and the computer resources that are being used, such as memory.
- `CrawlCommand`. This command type allows the user to start, pause and terminate the crawling process. The user can also skip a page using this type of command.
- `ExportCommand`. This command exports the encountered links to a file.

3.2.7 DATABASE MODULE

The database module handles all persistent data storage. Because the database is used in many parts of the system, it is set up in such a way that is easy to integrate into other modules. Figure 15 shows the design of the

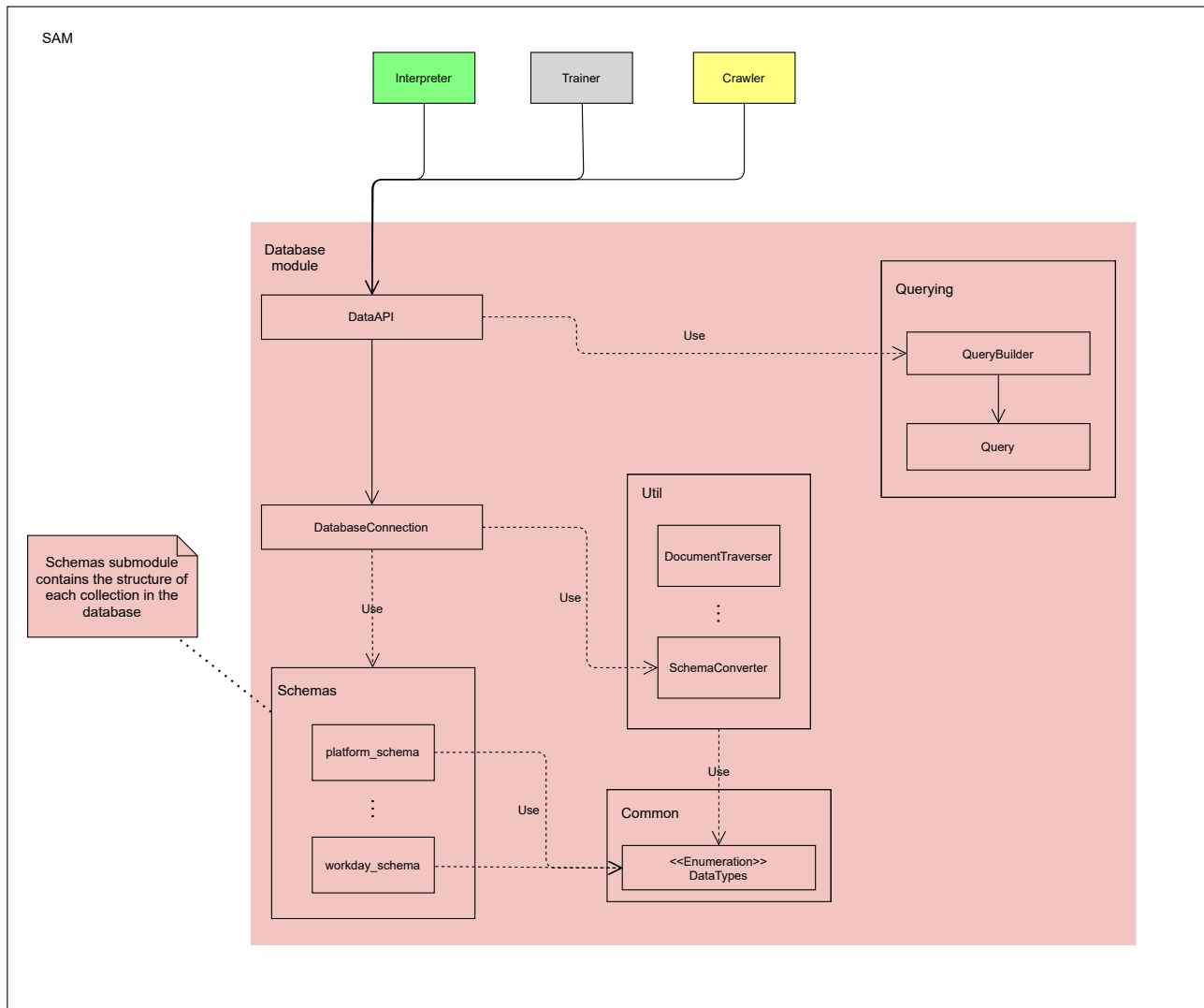


Figure 15: Class Diagram of the Database Module and the API.

database. The database has a DataAPI, which allows other modules, such as crawler, trainer and interpreter, to access the database. The module can be divided into the following logical components:

- The connection to the database
- The custom API
- The query builder

We will elaborate each of the logical components below.

3.2.7.1 The connection to the database

The `DatabaseConnection` class opens a connection using the database address and credentials. It verifies that the connection has opened correctly and will throw relevant exceptions when a connection could not be made. These errors are documented in the Software User Manual (SUM). After opening a connection, all the verification schemas are uploaded to the database. A verification schema essentially puts constraints on the contents of a collection. More specifically, a schema can be used to specify the required properties of documents along with their data-types. Each schema defined will first be converted to MongoDB's `$jsonSchema` format, and then it is sent to the database server such that the schema can be enforced.

3.2.7.2 The custom API

The custom API is exposed in the `DataAPI` class. It takes an instance of the `DatabaseConnection` class and uses that connection to execute queries. The `DataAPI` class does not contain any query methods. Each time the API is accessed, a `QueryBuilder` object is instantiated and returned. The Query builder is then used to construct the query. Once the query is constructed, the API will execute the query and return the result.

MongoDB can store binary data like regular data, but once it reaches a threshold size, binary data must be stored in a different way compared to regular data. To maximise the ease of using the API, it scans each insert query for binary data, before it is executed. If a document contains any binary data, the data is stored separately, and the data inside the document is replaced with a reference. When a document is retrieved, the opposite happens. References to binary data are replaced with the binary data itself. However, this is only done when the user requests it, as the operation increases the total execution time of the query. In this context, the user is the developer that writes the query.

3.2.7.3 The Query Builder

The `QueryBuilder` class contains methods to construct a query. Each method returns the instance of the `QueryBuilder` class, such that method calls can be chained. This allows for concise queries. The methods themselves are very similar to the methods of the original API in terms of usage, because there simply is no good reason to make them different. It exposes the default insert, read, update and delete functionality, along with the feature that strips and attaches binary data, as described in the previous paragraph. The query builder contains an `exec` method that needs to be called when a query is constructed. This will execute the query and return the result.

3.3 STATE DYNAMICS

Having addressed the logical model of SAM, we will now discuss the state dynamics of SAM. Several sequence diagrams illustrate multiple user actions and system behaviours. Each diagram is based on the URD [1] and includes a description.

3.3.1 INSERTING KEYWORDS

The user starts up SAM and will be asked on the start screen to fill relevant and blacklisted keywords. This can be done before the crawling has started.

Goal The user sets the keywords for the crawling session.

Precondition The user has started SAM and is on the start screen.

Postcondition The user filled in the keywords, which is saved by the system, and is able to start the crawler with the configuration applied.

User The user operating on the start screen.

Priority Must have

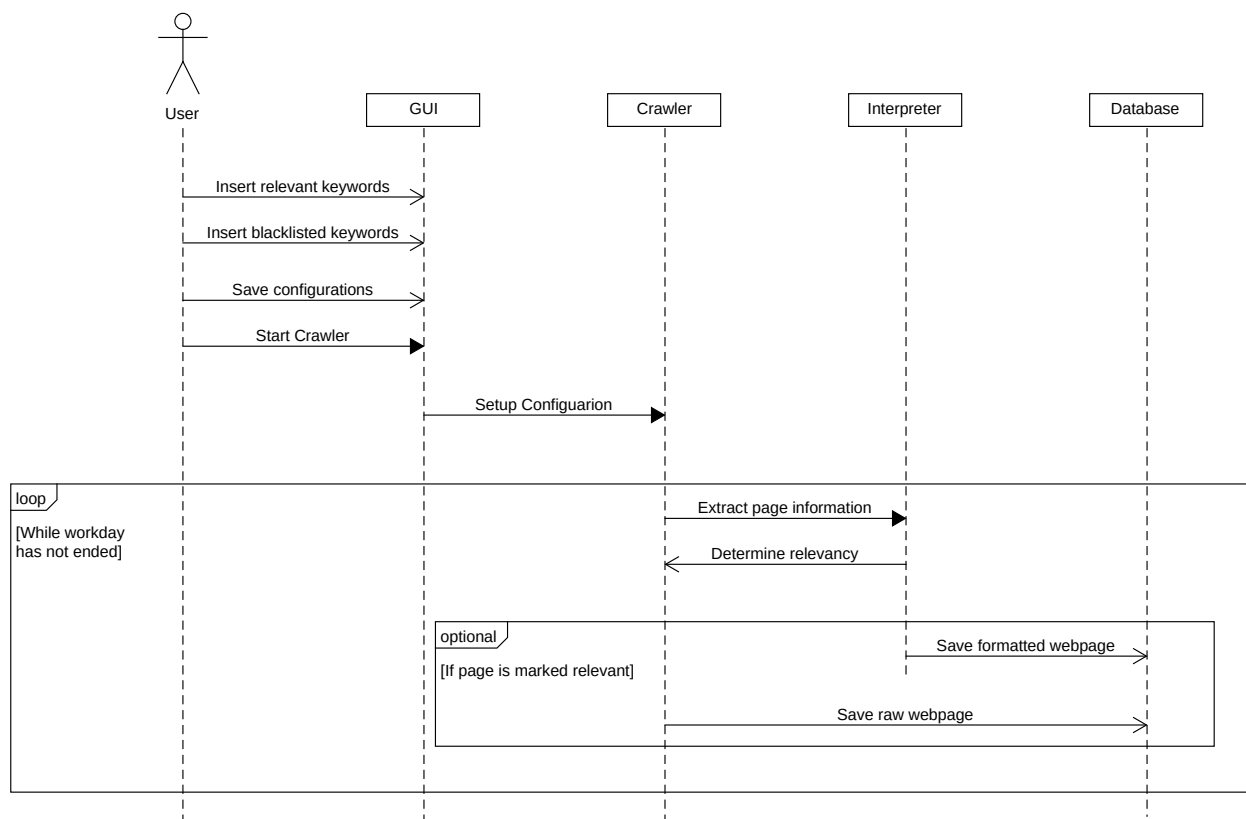


Figure 16: Sequence diagram for inserting keywords.

3.3.2 SAM STARTS AND IS TRAINED ON THE FIRST WEB PAGE

Refer to the sequence diagram in Figure 6. The user starts SAM and trains SAM on the first web page. The first web page that the user needs to train SAM on is either the front page or log-in page. If the platform structure is saved in the database, then the user is asked to verify that it is correct and adjust it if needed. If the platform structure is not saved in the database, then the user needs to specify the structure entirely from scratch.

Goal	The user starts SAM and trains it on the first web page.
Precondition	SAM is not running.
Postcondition	SAM is running and is trained on the first web page.
User	The general user that trains SAM
Priority	Must have

3.3.3 PAUSING SAM

The user wants to pause the crawler while SAM is crawling.

Goal	The user pauses the crawling session with a command.
Precondition	The crawler session is running.
Postcondition	The crawler is paused and not crawling anything.
User	The user inputting the pause command in the CLI.
Priority	Must have

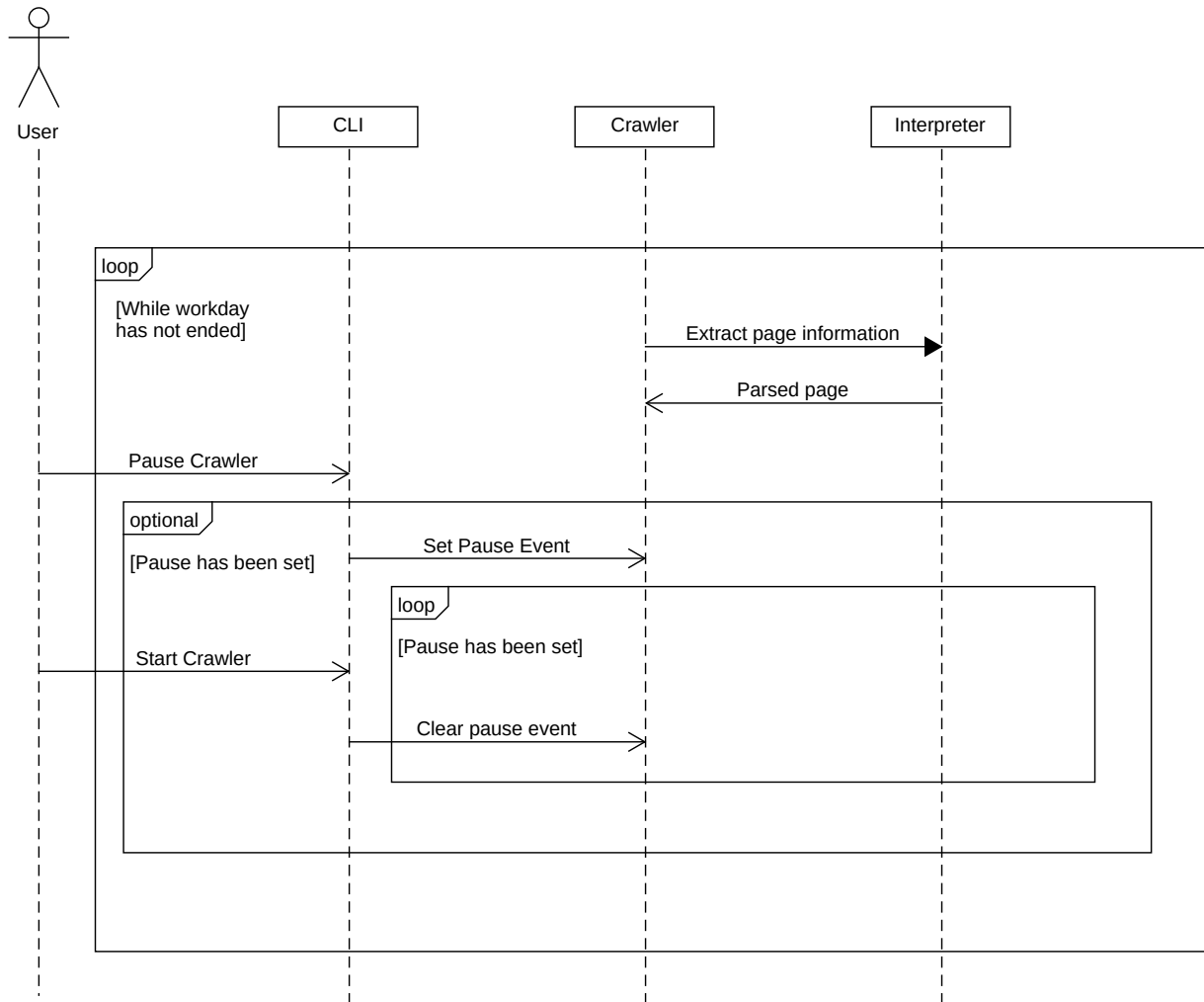


Figure 17: Sequence diagram for pausing the crawler.

3.3.4 TERMINATING SAM

The user wants to terminate the crawler while SAM is crawling.

Goal	The user terminates the crawling session with a command.
Precondition	The crawler session is running.
Postcondition	The crawler is terminated and the instance of the Tor Browser is closed.
User	The user inputting the terminate command
Priority	Must have

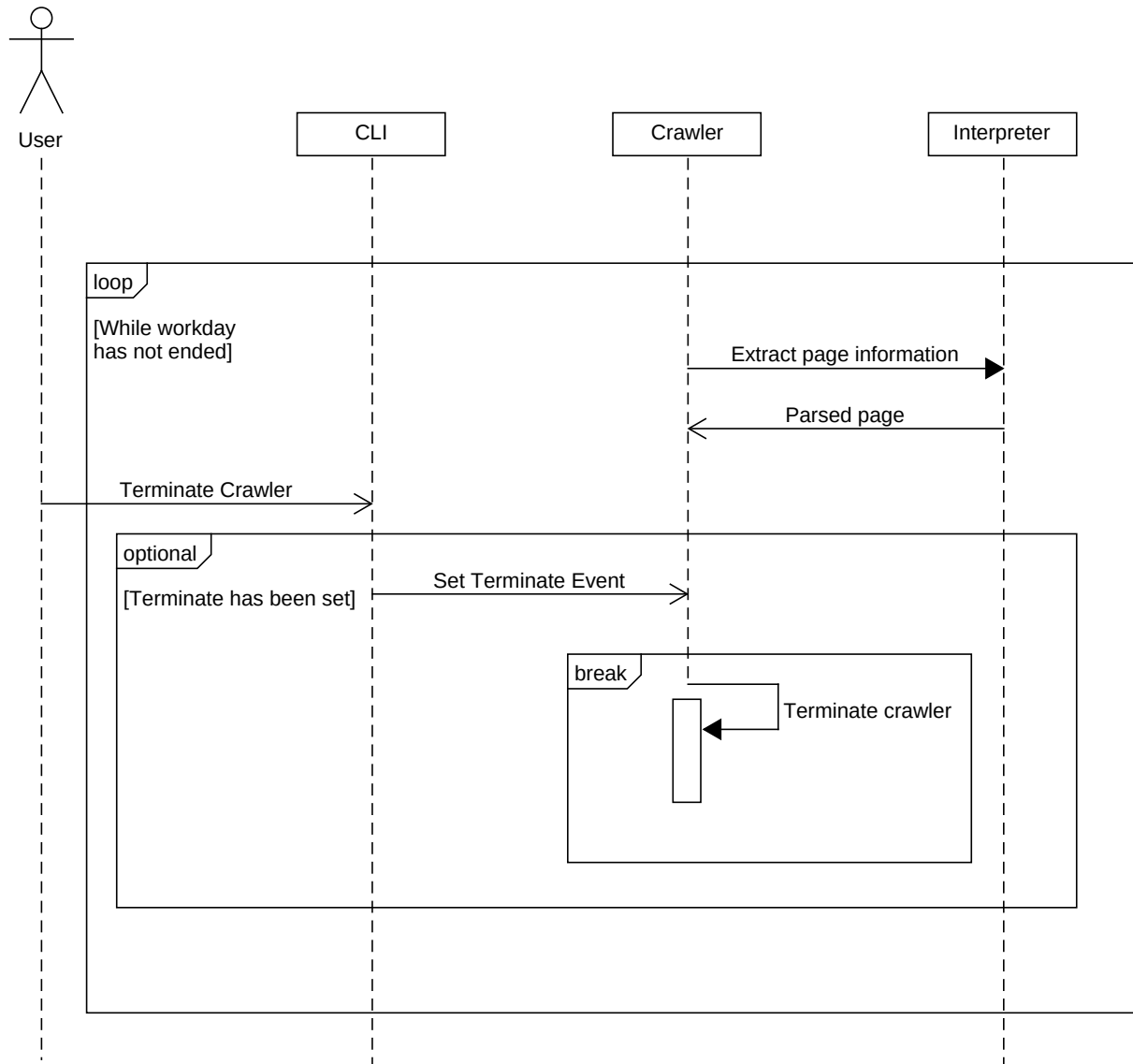


Figure 18: Sequence diagram for terminating the crawler.

3.3.5 GENERATING A SCHEDULE

The user wants to create a schedule for the workday of the crawler. The crawler will follow this schedule during crawling that has been inserted by the user.

Goal	The user schedules the crawler.
Precondition	The user has started SAM and is on the schedule tab of the start screen
Postcondition	The user has set the schedule parameters and is able to start the crawler with these parameters applied.
User	The user inputting the schedule.
Priority	Must have

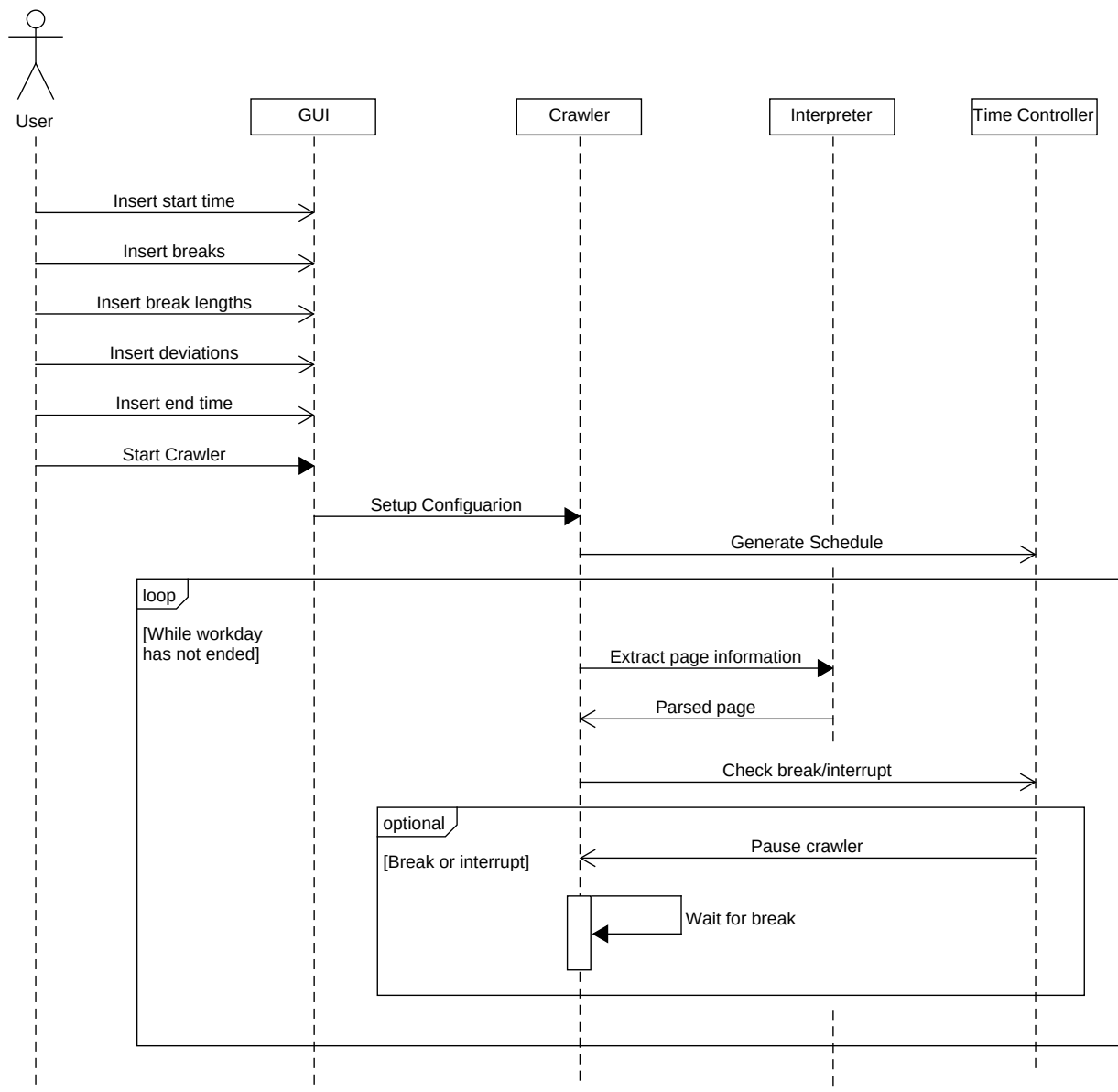


Figure 19: Sequence diagram for inserting and handling breaks.

3.3.6 CRAWLER LOG IN

The crawler will sometimes be asked to log in on the website before continuing browsing the website. The crawler will then sign in using the configuration settings inputted by the user.

Goal	The crawler signs in the page and can continue browsing the forum.
Precondition	The crawler is on a log in page of the forum.
Postcondition	The crawler is signed in using the account of the user.
User	The general user that inserted the log in information
Priority	Must have

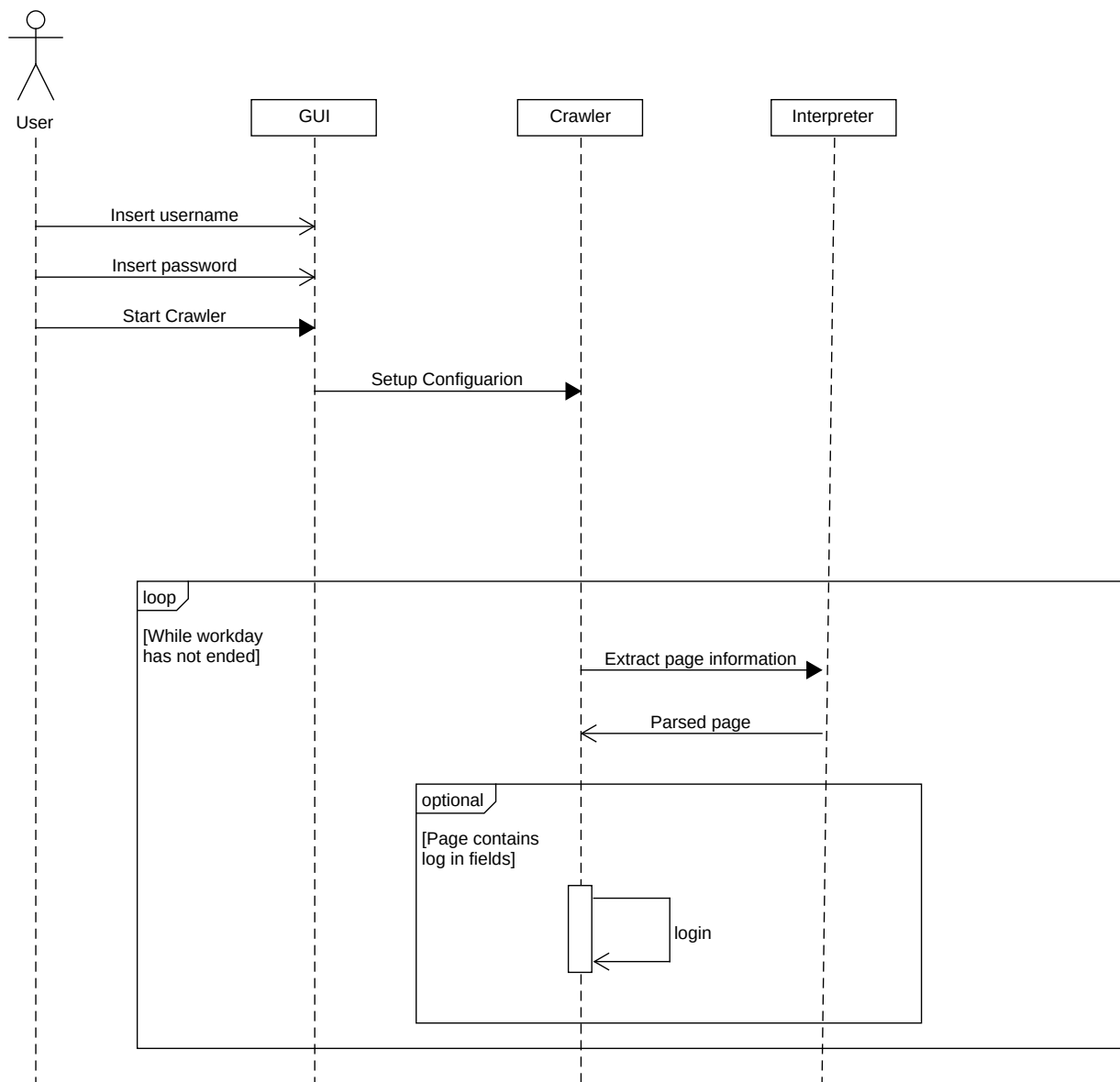


Figure 20: Sequence diagram for acquiring log in information and crawler logging in.

3.3.7 CRAWLER STORING A RELEVANT WEB PAGE

When the crawler encounters a relevant web page, it should store the page into the database.

Goal	The information on the web page is formatted and on the database.
Precondition	The crawler is on a relevant web page that do not contain any blacklisted keywords.
Postcondition	SAM stored the data of the web page on the database.
User	Not applicable
Priority	Must have

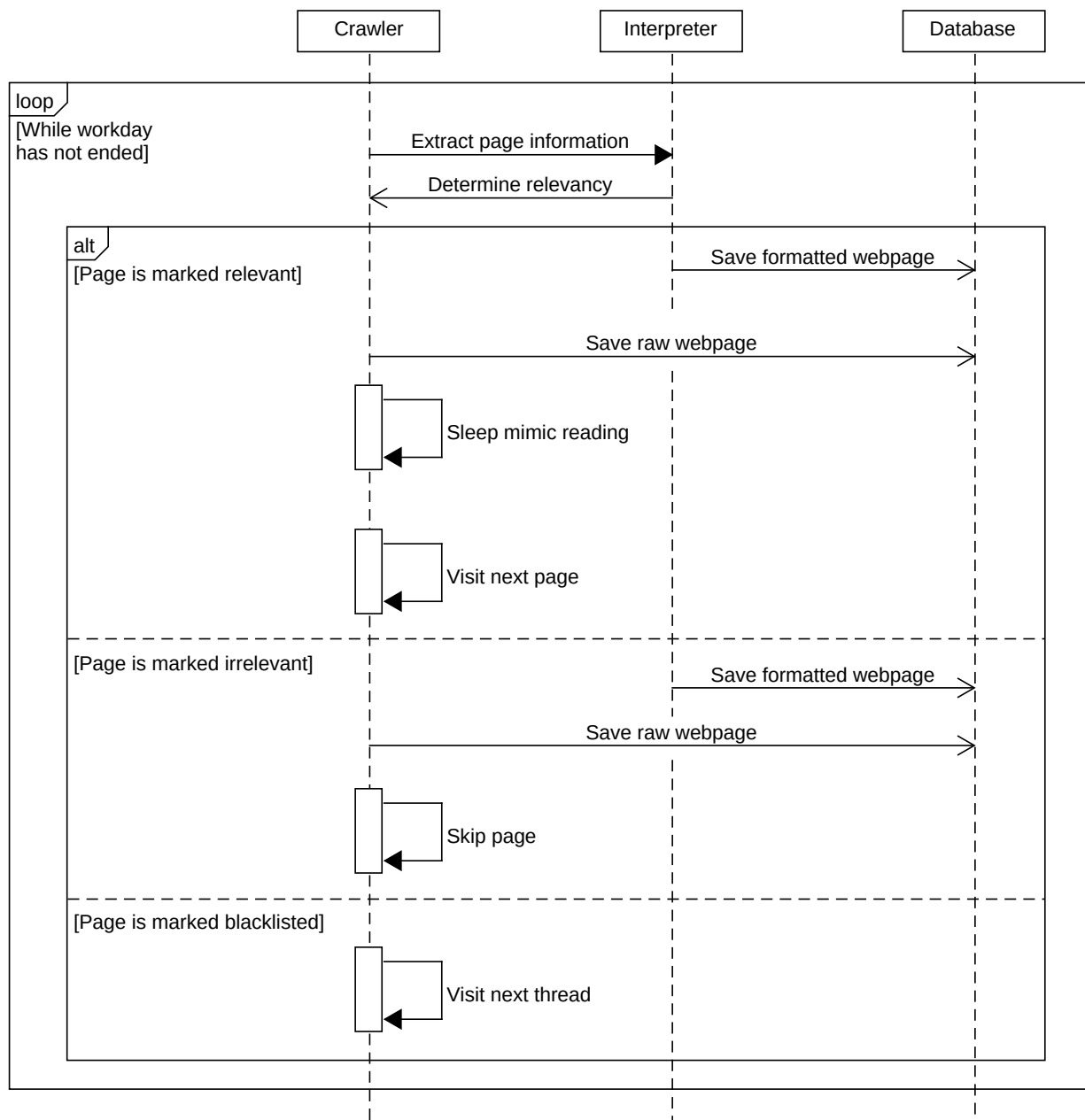


Figure 21: Sequence diagram for handling pages with different relevancy.

3.3.8 CRAWLER ASKS USER TO SOLVE CAPTCHA

The crawler can encounter various CAPTCHAs. These CAPTCHAs are hard to solve for the crawler, and thus the crawler will ask the user to solve the CAPTCHA.

Goal	The CAPTCHA is solved by the user and the crawler can continue crawling.
Precondition	There is a CAPTCHA on the webpage the crawler is currently on.
Postcondition	The CAPTCHA input field is filled in by the user.
User	The general user that solves the CAPTCHA
Priority	Must have

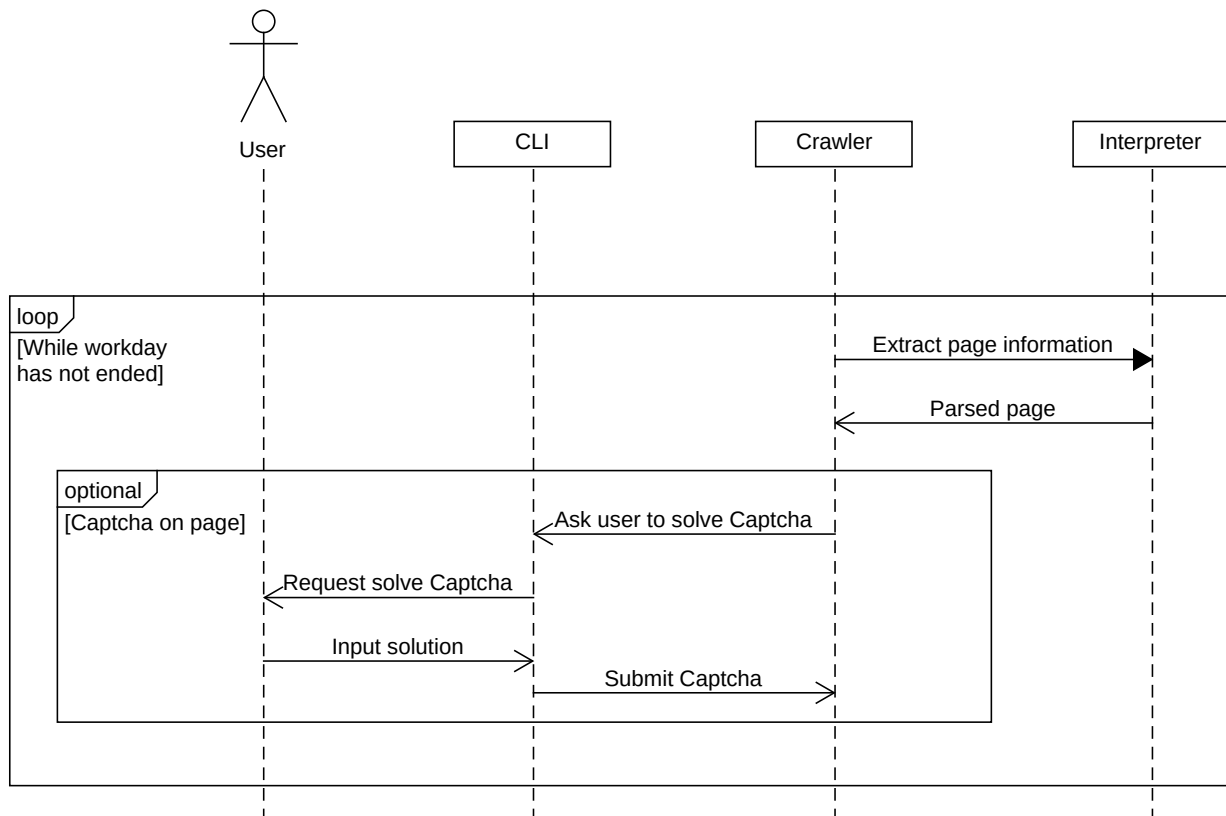


Figure 22: Sequence diagram for handling and solving CAPTCHAs.

3.4 DATA MODEL

SAM has a data model to structure the information it retrieves from web pages. Figure 23 provides a high-level overview of this data model. The entities in the data model are stored in the database, along with their properties. The properties of the entities of the data model are explained below.

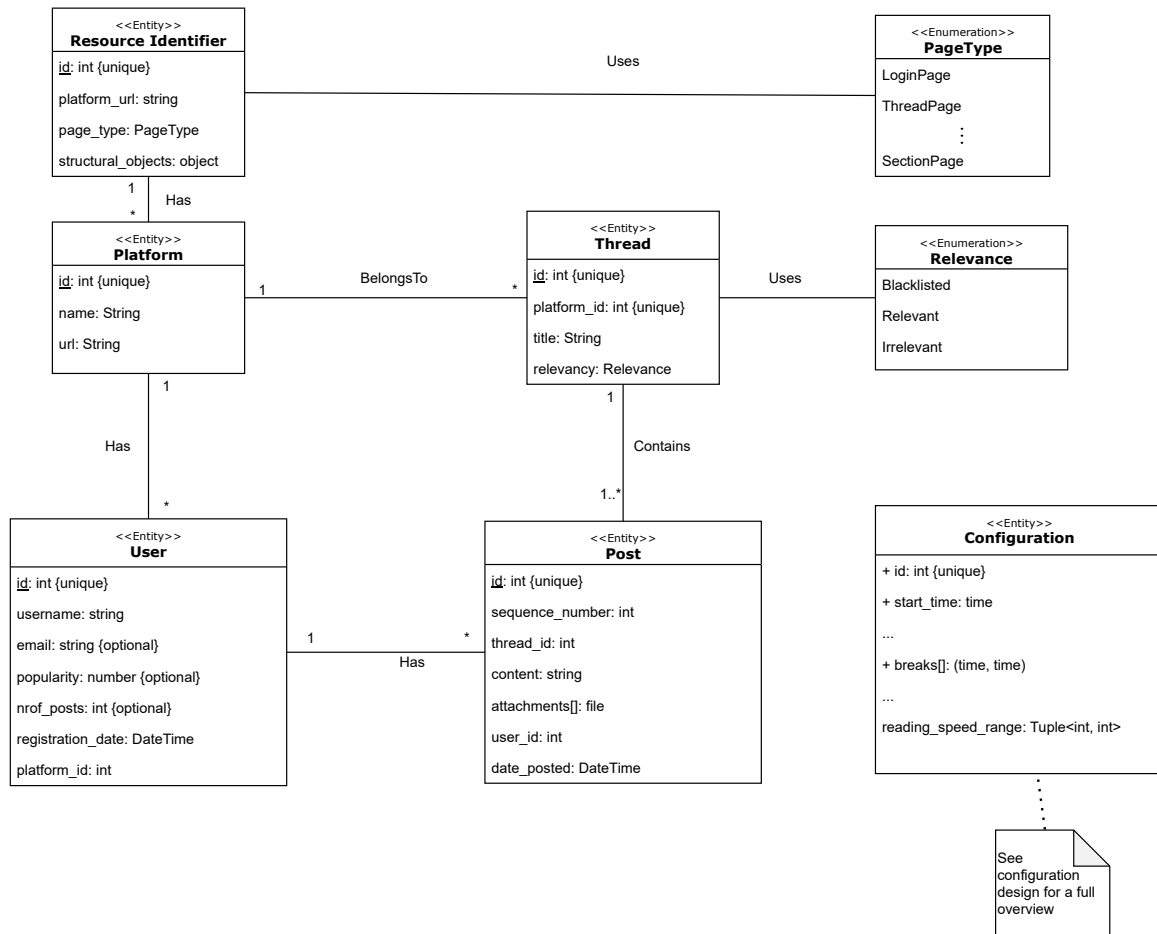


Figure 23: Diagram indicating the data model of SAM.

Figure 23 shows the ER-diagram. The diagram is described by the following behaviour:

- SAM saves every platform that it crawls into the database. A platform gets a unique ID. The URL and name of the platform are saved. Within the platform, it saves the threads and their posts to separate collections.
- Threads are saved with their relevancy, which is determined by the keywords specified in the GUI, as well as their title and the platform on which they were found. The relevancy is stored as an enumeration.
- Posts are also saved. Each post has a thread it belongs to, content and a user that created it. The date at which the post was made is also saved.
- The users making posts on the platform are stored as well. They are linked to a platform and their information is stored along the number of posts they made on that platform.
- A platform also has resource identifiers for their web pages. The type of page alongside the structural objects are saved to the database. The type of page is saved as an enumeration.
- Every configuration, with their added workday schedule, is also saved in the configuration collection. The full configuration design is explained in Section 3.2.4
- Relations are not described, as the database is non-relational. The only relations present are between a platform and their users/threads.
- The collections that are described above are defined by database schemas, defining the fields that a collection has and of what type they should be. The combination of schemas defines the database structure and this structure can be seen in Figure 23.

3.5 EXTERNAL INTERFACE DEFINITIONS

SAM is a web crawler, and thus relies on a web browser to execute the crawling procedure. Since SAM crawls the dark web, the first external interface is the Tor browser. As SAM crawls, it collects data. This data has to be stored, and thus our second and last external interface is that of the database, namely MongoDB.

3.5.1 TOR BROWSER

Tor is known for the way it redirects and encrypts an internet connection, thus enabling the use of the dark web. For this reason, SAM must interact with Tor. Besides, CAPTCHA solving has to be done manually, therefore SAM must be interacting with a Tor browser such that this CAPTCHA can be solved.

As mentioned before, SAM relies on this web browser to execute the crawling procedure. For communicating with the Tor browser, the `tor-browser-selenium` Python module is used [14]. This exposes a `TorBrowserDriver` class for controlling the browser. In this controlled browser, the user of SAM can solve CAPTCHAs.

3.5.2 MONGODB DATABASE

A MongoDB [11] database is a way to store data. MongoDB is special because it is unstructured and non-relational, and stores data in a JSON-like format. This means a regular table is called a collection, which stores JSON objects instead of the regular row/column inputs.

To still give some relations between collections, MongoDB allows the implementation of foreign keys. IDs are of the `ObjectId` type, and relations can be formed with these types as seen in Figure 24b.



Figure 24: Example features of a MongoDB database.

Figure 24b shows an entry in the `posts` collection containing `ObjectIDs` to the `users` collection through `user_id` and to the `threads` collection through `thread_id`. These `ObjectIDs` are specified as `_id` in the now related collections. Querying `.include('thread_id', 'threads')` on a `posts` query includes the entries of the `threads` collection where the `thread_id` in `posts` is the `_id` in `threads`. Under the hood, the API will just retrieve the posts in a separate query and use it to replace the corresponding `ObjectID`'s.

MongoDB comes with a query language. Like SQL databases have a MySQL language. MongoDB has the MongoDB language. Once connected to the database, a collection can be selected. On that collection, MongoDB offers various functions to interact with the database:

- `insert_one({item: value})`: Insert data into the database. If the collection specified has a schema, the required items have to be specified. The key of the collection is automatically generated if not specified.
- `insert_many(list)`: inserts all `{item: value}` dictionaries into the collection.
- `find(query)`: finds values given a certain query. For instance, executing `find({'name': 'Jeroen'})` on a `people` collection finds all fields in `people` for which the `'name'` is `'Jeroen'`
- `find_one(query)`: Similar to `find`. However, this function only returns the first result.

The MongoDB query language comes with many more functionalities; the functions listed above are the functions our design will use.

3.6 DESIGN RATIONALE

We have made various design decisions during the project, including the browser which is used during the crawling, the back-end language and the front-end framework. The design rationale will be explained by comparing available technologies and techniques within the scope of this project.

3.6.1 TOR-BROWSER-SELENIUM

The first design choice we made was which browser to use while crawling. For this purpose, we decided to use the Tor browser, because it enables anonymous visits to websites. Moreover, we determined to use a portable framework to automate the Tor browser. Thus, We chose the tor-browser-selenium library, also known as tbselenium, since it is the only library providing APIs that allow the Tor browser to be controlled without user interaction. Another advantage of tbselenium is that it uses GeckoDriver, a browser engine that enables manipulation of the browser's behaviour. Finally, tbselenium produces traffic identical to Tor browser and. This means that web servers consider this traffic to be legitimate, preventing the crawler from being identified as a bot.

3.6.2 BACK-END LANGUAGE

The back-end processes the logic related to functionalities of the product and serves as the core part of SAM. There are different server-side languages we could choose from, such as PHP, Ruby, Python and Java. However, we decided to use tbselenium, which is a library exclusive to Python. Additionally, most of the team had adequate pre-knowledge of Python. Therefore, we chose Python as the back-end language for SAM. Python also has the advantage that it has a massive list of libraries related to parsing web pages. Many of them are used during our project, and we will discuss the details in Section 3.6.8.

3.6.3 FRONT-END LANGUAGE

The front-end of SAM consists of the GUI and the CLI. Each of these interfaces required decisions to be made.

We decided to use Python to implement the CLI since it is the same language as the back-end development. Using the same language can ensure that there are fewer compatibility issues.

On the other hand, there are two possibilities for GUI development. One is to build a standalone application with Electron [16], which is based on JavaScript, HTML and CSS. Another possibility is to build a GUI with Python libraries. Initially, we considered continuing with Python to remain consistent with back-end development. However, the core functionality of the GUI is to enable user interaction, and a fast response time is required. To have a better looking and user-friendly GUI, Javascript has more advantages than Python. It is faster than Python for the reason that it can be run immediately within the browser. Due to these reasons, we decided to switch from Python to Electron to produce a stand-alone desktop application. Another advantage of using Electron is that it is a framework that packages front-end code, usually HTML, inside an application based on Chromium. Thus, Electron is able to use front-end language HTML by parsing the HTML element when training the platform.

Apart from Electron, React was used to make the GUI application easier and more fluent. React is an open-source front-end JavaScript library for building User Interface (UI) components. We chose React considering it provides the best possible rendering performance and allows us to break down the complex UI into simpler components. These components can then be used where needed and prevent code duplication. It also gives a better overview, and files get less cluttered.

3.6.4 OPERATING SYSTEM

For the environment that SAM uses, we chose Linux as our operating system due to the tbselenium plugin that is only supported on Ubuntu and Debian. Ubuntu is one of the most commonly used Linux distributions and is slightly easier to use and install compared to Debian. For this reason, we decided to use Ubuntu instead of Debian. Although there are many versions of Ubuntu to choose from, we decided to use the latest stable release version, which was Ubuntu 20.04 at the time of this project.

3.6.5 DATABASE

The client requested that we use a No-SQL database, preferably ElasticSearch or MongoDB. For our project, MongoDB is the best fit for multiple reasons. Firstly, ElasticSearch is primarily a search engine and not a database, whereas MongoDB is primarily a database. For a fair comparison, we have installed both databases and connected to them using Python. We were able to store and retrieve data in MongoDB faster than in ElasticSearch. As a result, MongoDB was found to be more user-friendly. Furthermore, MongoDB's learning curve is not as steep as that of ElasticSearch. This is reflected in the API documentation for both options, located at <https://docs.mongodb.com/drivers/pymongo/> and <https://elasticsearch-py.readthedocs.io/en/v7.13.2/api.html/>. Moreover, MongoDB has a larger user-base, and hence there are more online resources available than for ElasticSearch. These observations have led us to conclude that MongoDB is the best choice for a database in our situation.

3.6.6 SEPARATION AND MODULARITY

Another important design decision we made is to apply modular programming in our back-end development. Modular programming separates the functionality of a program into several different independent modules such that each module contains everything necessary to execute only one aspect of the desired functionality.

For that reason, although `CARONTE` used to parse web pages while crawling them, we chose to put the parsing of the web pages and the crawling into separate modules, which are the crawler module and the interpreter module. The strategy is also applied in other functionalities of SAM, as we want to keep the complexity of one module as low as possible. Furthermore, it makes collaboration easier for the team, because all modules can be implemented simultaneously without interfering with each other. Additionally, each module of SAM could be modified easily and the software could be expanded without requiring knowledge about the implementation details of other modules.

3.6.7 CONTINUOUS INTEGRATION AND DEPLOYMENT

A continuous integration (CI) pipeline was set up to increase the quality and consistency of the code produced. We chose GitLab as the CI environment because it is the repository manager that we use for collaboration. Moreover, GitLab CI is easy to use and supports cloud-native development, which fits teamwork well.

The CI pipeline is set up in the GitLab environment with two distinct steps:

1. The first step, styling, ensures that every Python file in the project is styled according to the Flake8 [18] guidelines. This creates consistency in layout and programming style. Whenever a file is not styled perfectly, warnings will be shown in the GitLab pipeline interface. The pipeline will not fail to ensure that the next step can still be executed.
2. The second step, testing, runs all of the Python unit test files in the repository. Both of these steps run in their respective Docker environments on two Ubuntu 18.04 LTS servers, of which one is hosted at the TU/e. It is being run on two servers to ensure maximum efficiency since these servers are available.

The steps mentioned above are only applicable for the back-end of the project and not for the front-end part due to the fact that most of the front-end is written in JavaScript, and our continuous integration system only considers code written in Python. Moreover, fewer problems were expected concerning the SEP coding requirements, given the front-end part is significantly smaller. Both the front-end and the back-end were checked on a weekly basis using Understand [19], an integrated development environment (IDE) that enables static code analysis. We chose Understand because it is also used by the supervisors to check the code according to the given guidelines. Executing this check once a week meant that the requirements were up-to-date and no surprises with respect to the complexity of the code or the dependencies had to be fixed last-minute.

3.6.8 PACKAGES IN PYTHON

We used the following Python packages for SAM:

- ewmh: The ewmh package [20] is an implementation of the Extended Window Manager Hints for Python. It allows SAM to check whether a certain window in the Ubuntu environment is focused. This is essential as, since the Tor Browser must be focused in order to save a file. Alternatives like libwnck and xdotool have been examined, but the only package that was found to be working reliably was ewmh.
- pyautogui: The pyautogui library [21] allows SAM to mimic keyboard inputs on a system level. This library is needed as tbselenium is limited to keyboard inputs within the browser, and SAM must interact with system windows to save the web page. Pyautogui, along with ewmh, allows SAM to save a web page using the built-in “save as” functionality of the Tor Browser, which saves a web page as a local page that can be reconstructed.
- lxml: The lxml XML toolkit [22] is a Pythonic binding for the C libraries libxml2 and libxslt. It allows for the handling of XML and HTML files. The advantage of the lxml package is that it is easy to use and extremely fast when parsing large documents. The trainer module uses the etree module in lxml and creates XML/HTML elements with their sub-elements to manipulate an HTML or XML file.
- BeautifulSoup4: The BeautifulSoup [23] library is used for pulling data out of HTML and XML files and provides idiomatic ways of navigating, searching, and modifying the parse tree. The trainer module uses it to extract outer HTML elements and retrieve the content of classes of the root HTML element in a web page.
- pymongo: The PyMongo library [24] is the official MongoDB driver for Python. The database module uses it to communicate with the database.
- tld: The tld library [25] could extract the top level domain (TLD) from the URL given. The interpreter and trainer modules use it to parse the received URL. It helps to transfer a web page URL into a platform URL. For example, `https://canvas.tue.nl/courses` is reduced to `canvas.tue.nl`.
- websockets: websockets [26] is a library used to build WebSocket servers and clients in Python. The trainer module uses it to connect to the GUI.
- flake8 and flake8-polyfill, pylint and pyflakes: The flake8 library [18] is a library used for code checking based on coding style (PEP8). The library flake8-polyfill [27] provides some compatibility helpers for Flake8 plugins. Pyflakes [28] is a library that checks Python source files for errors, and pylint [29] is a Python static code analysis tool looking for programming errors. All four packages are used in the CI pipeline to ensure consistency in layout and programming style.
- pytest, pytest-cov and pytest-datafiles: The pytest library [30] is a framework that users can apply to write unit tests in python. It is useful for both small and complex functional testing. The pytest-cov package [31] is a pytest plugin that produces coverage reports; pytest-datafiles [32] is a pytest plugin used to create a temporary directory containing a pre-configured set of files and/or directories. All three packages are used in the CI pipeline to write and run the Python unit test files.
- xenon: The xenon library [33] monitors the code complexity based on Radon, a tool that computes various metrics from the source code. It is used to install all the packages on the CI pipeline.
- pyyaml: The pyyaml package [34] is a YAML parser and emitter for Python. The package is used to read `.yaml` files. YAML is the human-readable data-serialisation language. SAM uses a yaml file for the configuration.
- python-dateutil: The package python-dateutil [35] library gives extensions to the standard datetime module in Python. The datetime in the package is used in interpreter module to convert strings into a datetime object.
- tbselenium and selenium: The tbselenium library [14] automates Tor Browser with Selenium and selenium is a library that automatically interacts with Firefox-based web browsers from Python. As is explained in Section 3.6.1, both of the packages are used in crawler module.
- termcolor: The termcolor package [36] includes ANSI Colour formatting for output in terminals. The package is used in the CLI module to provide different colours to different types of output messages.
- pytz: The pytz package [37] includes the Olson timezone database, a compilation of information about the world's time zones, into Python and supports different time zones. The package is used in interpreter module to identify the given timezone.

- coverage: The coverage package [38] is a tool for measuring code coverage of Python programs and is used for creating a code coverage report.
- rich: The rich package [39] is a library for writing rich text with colour and style to the terminal, we use it for CLI module to display the progress bar when waiting for the workday to start.

4 FEASIBILITY AND RESOURCE ESTIMATES

This section discusses the resources required to run SAM, as well as resources required for developing SAM. Moreover, this section discusses the performance of SAM, how this performance is defined and how the performance was measured.

4.1 RESOURCE REQUIREMENTS

Various computer resources are needed to run and develop SAM. Requirements for three types of machines are given separately, but these can coincide:

- Development machine: for developing SAM
- Client machine: for running one crawler instance of SAM
- Database machine: for running the database of SAM

These resource requirements were determined by looking at the resources consumed by SAM during its execution, coupled with educated guesses and some margins.

Note: the memory and disk space requirements do not include the amount needed to run the operating system.

4.1.1 DEVELOPMENT MACHINE REQUIREMENTS

CPU	Any capable of running the operating system below
Operating System	Ubuntu 20.04 LTS
Memory	Minimum 2GB
Disk Space	Minimum 2GB
Network	At least 1 Mbit/s for responsive usage
Browser	Tor, version at least 10.0.16
Other Software	Node.js latest LTS version, Python 3.9, MongoDB Compass, GeckoDriver

4.1.2 CLIENT MACHINE REQUIREMENTS

CPU	Any capable of running the operating system below
Operating System	Ubuntu 20.04 LTS
Memory	Minimum 2GB
Disk Space	Minimum 2GB
Network	At least 1 Mbit/s for responsive usage
Browser	Tor, version at least 10.0.16
Other Software	Python 3.9, MongoDB Compass, GeckoDriver

4.1.3 DATABASE MACHINE REQUIREMENTS

CPU	Any capable of running the operating system below
Operating System	Any operating system supported by MongoDB (see https://docs.mongodb.com/manual/installation/#supported-platforms)
Memory	Minimum 2GB
Disk Space	Minimum 4GB
Network	At least 1 Mbit/s for responsive usage

4.2 PERFORMANCE

Measuring the performance of SAM consists of several aspects. First, the performance is defined using metrics. Next, the setup of the performance test is explained. Lastly, the results of the performance test are given.

4.2.1 PERFORMANCE METRICS

To measure the resource usage of the processes used by SAM, we use the following two metrics.

- The CPU usage of the process as a percentage of a single core of the CPU. Note that because the percentage is of a single core, the CPU usage of a process running on a multi-core processor can be over 100%.
- The Residual Set Size (RSS) of the process. The RSS is the amount of memory the process holds in main memory (RAM). The RSS includes shared libraries that are in memory. Therefore, summing the RSS of multiple processes may yield a higher number than the actual memory needed to run those multiple processes.

We use these two measures because they are provided by the testing tool we decided to use. Our choice for this testing tool is explained below.

4.2.2 TEST SETUP

SAM consists of many processes. For example, the GUI has a start screen process before crawling and a training screen process that is created when training is required. Additionally, there are processes related to the Python back-end, the GeckoDriver and the Tor browser. To measure all these processes that are created and destroyed at different times, we decided to use the `procpath` Python module [40]. We chose this module because it provides a simple way to specify which processes should be measured. In particular, the module is capable of finding child processes. The Tor browser and GeckoDriver are both child processes of the Python process used by SAM. The Python process, however, is not a child process of the GUI. All processes SAM uses can therefore be tracked by considering the child processes of the GUI and Python processes. The database was not considered for this test, because it runs on a separate machine. To do this, the following commands were run, simultaneously and in separate shells.

- `procpath record -i 1 -d sam.sqlite '$..children[?("sam" in @.cmdline)]'`
- `procpath record -i 1 -d python.sqlite '$..children[?("python3" in @.cmdline)]'`

These commands record the resource usage for all processes with either `sam` or `python3` in their `cmdline` (the command that started the process), and their children processes. The option `-i i` specifies that the resource usage of the processes is measured once per second.

After recording the data, we generated plots using the built-in visualisation functionality of `procpath`. This was done via the following commands:

- `procpath plot -d sam.sqlite -q cpu -w 4 -p S1,S2,... -f sam-cpu.svg`
- `procpath plot -d sam.sqlite -q rss -w 4 -p S1,S2,... -f sam-rss.svg`
- `procpath plot -d python.sqlite -q cpu -w 4 -p P1,P2,... -f sam-cpu.svg`
- `procpath plot -d python.sqlite -q rss -w 4 -p P1,P2,... -f sam-rss.svg`

These command generate plots of the percentage of CPU and amount of RSS used by the different processes. The `-w 4` specifies that a moving average is computed with a window of size 4. Additionally, the `-p S1,S2,...` and `-p P1,P2,...` options specify which GUI and Python process IDs should be included in the plot. This option was used to remove the following processes.

- Processes that were listed in the legends of the RSS plots but not in the legends of the CPU usage plots. Though the process IDs were listed in the legends of the RSS plot, there were no corresponding lines visible in the RSS plot. Therefore, they added no useful information.
- For the Python part, some processes were recorded that used Python but were not part of SAM. This includes `procpath` processes.

Test machine

All tests were performed in Ubuntu 20.04. The machine had an Intel(R) Core(TM) i7-7700HQ CPU and 8GB of RAM. Testing was done on the `crdclub.su` web page. Note that the tests were not run inside a virtual machine.

4.2.3 RESULTS

Figure 25 and Figure 26 show the results of the tests. When a training screen is opened, the main GUI processes become active, and new processes are created for the training window. This can be seen from the spikes in Figure 25a and the small lines in Figure 25b. These lines are the training screen processes, which are created when training is needed and destroyed when the user confirms the structure.

The large CPU usage of the process with PID 11115 after around 8:10:40 (Figure 26a) is striking. At this time the crawler has finished the training session, and starts reading the front page. The process with PID 11115 is the 'Web Content' process of the Tor browser. One likely reason for the high CPU usage is the large amount of GIFs that are present on the `crdclub.su` front page. Large CPU usage of the 'Web Content' process is also seen earlier in time; this was during the training of the front page of the platform.

The plots support the resource requirements stated in Section 4.1. Adding the RSS usage of all the processes used by SAM yields around 2GB. As mentioned before, because summing the RSS usage of processes counts shared libraries multiple times, the actual memory usage will most likely be lower. The peak single-core CPU usage of SAM in our tests was around 160%. Since the test machine CPU had 8 cores, this comes down to around 20% CPU usage. Noting that this is the peak usage, we can conclude that SAM does not heavily use the CPU. Our tests also made clear that the CPU usage depends a lot on the web page that is crawled. The 'Web Content' process ranges from around 5% single-core CPU usage to around 50%.

Lastly, one should keep in mind that the test only measured the resource usage of SAM for roughly the first seven minutes of operations and ended shortly after the training phase was completed. We suspect that the training phase is the most resource intensive part of SAM's operation, and therefore, this phase is an accurate representation of its entire operation. However, it could be possible that, for example, the RSS usage of SAM slowly increases over time during its operation. A complete test of SAM's resource usage could not be done since, at the time of writing, SAM is non-functional after the training phase.

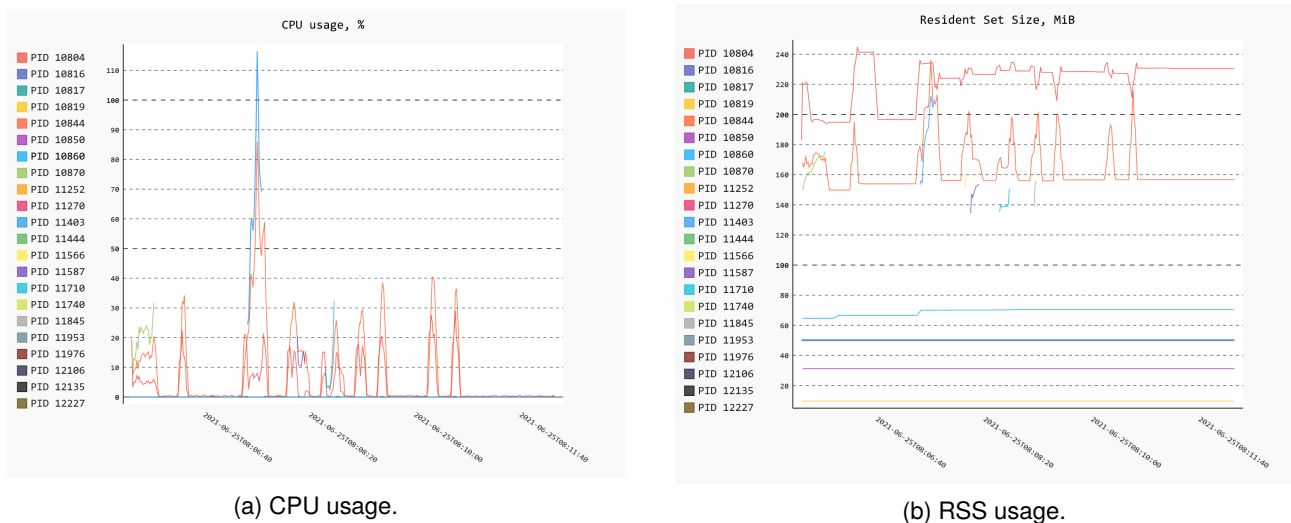
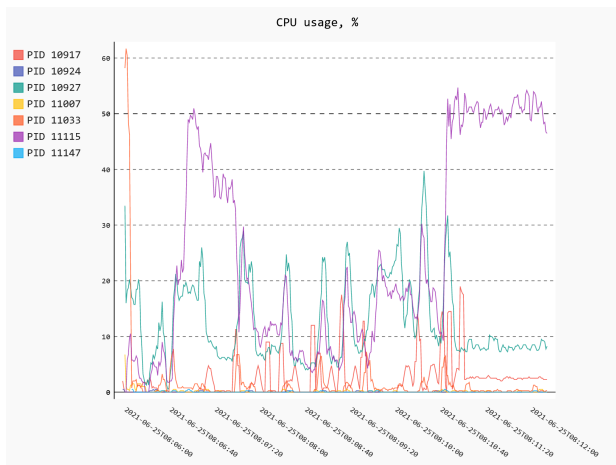
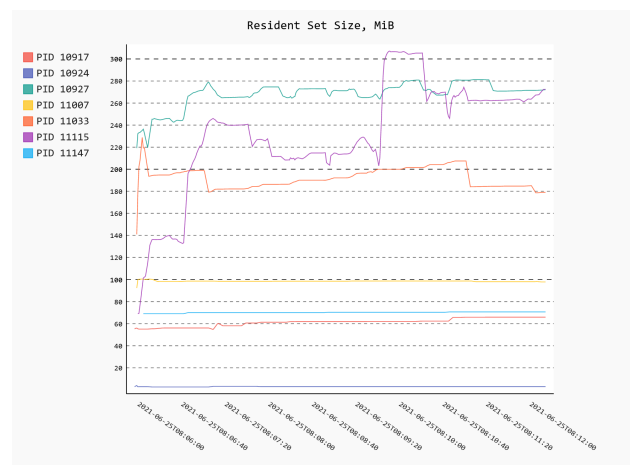


Figure 25: GUI child processes.



(a) CPU usage.



(b) RSS usage.

Figure 26: Python child processes, including the GeckoDriver and Tor browser.